Computer Communications 217 (2024) 166–182

Contents lists available at ScienceDirect

Computer Communications

journal homepage: www.elsevier.com/locate/comcom

Mobility aware and energy-efficient federated deep reinforcement learning assisted resource allocation for 5G-RAN slicing

Yaser Azimi [a], Saleh Yousefi [a,*], Hashem Kalbkhani [b], Thomas Kunz [c]

[a] Computer Engineering Department, Urmia University, Urmia, Iran
[b] Faculty of Electrical Engineering, Urmia University of Technology, Urmia, Iran
[c] Systems and Computer Engineering Department, Carleton University, Ottawa, Canada

ARTICLE INFO

Keywords:
RAN slicing
5G
Deep reinforcement learning
Mobility awareness
Power allocation
Federated learning

ABSTRACT

Network slicing is one of the foundations for the realization of 5G and beyond. However, due to the mobility of the users and the network dynamics, flexible and efficient radio access network (RAN) resource slicing is still a challenge. In this paper, we allocate the power and radio block (RB) resources in a multi-RAN scenario to the users of both rate-based and resource-based slices. We propose a mobility-aware and energy-efficient federated deep reinforcement learning-assisted resource allocation (ME-FDRL-RA) method for RAN slicing in a large multiple RAN environment. ME-FDRL-RA includes both federated deep reinforcement learning (FDRL) and deep learning (DL) models as follows: Stacked and bidirectional long-short-term-memory (SBiLSTM), allocate resources to slices on a large time-scale. Additionally, federated advantage actor-critic (F-A2C) allocates resources on a small time-scale and speeds up convergence. Moreover, to solve the optimization problem of determining the required resources for the users of slices, we propose an efficient iterative algorithm called the interference-aware and energy-efficient power allocation (IA-EPA) method. According to simulation results, ME-FDRL-RA outperforms competitive methods in terms of convergence speed, computational complexity, energy efficiency, and the number of accepted users while addressing the challenges of user mobility and maintaining a desirable degree of inter-slice isolation.

1. Introduction

The fifth-generation (5G) and beyond 5G (B5G) wireless networks are expected to provide a variety of heterogeneous services and applications with regard to user demands like massive connectivity, low latency, improved energy efficiency, and high transmission rates. Due to the increasing mobile data traffic, a high density of mobile users, heterogeneous users, and decentralized structures, B5G networks try to improve the quality of service (QoS) and the quality of experience (QoE) of the users [1]. By 2027, 4.4 billion subscribers will have access to 5G/B5G technologies, according to [2]. Also, its economic value over the next decade will be between 1.4 trillion dollars and 1.7 trillion dollars.

Although 5G/B5G supports a wide range of applications to fulfill the requirements of each application, International Telecommunication Union (ITU) generally classifies the applications of the 5G network into three categories: Enhanced mobile broadband (eMBB) communications, ultra-reliable low-latency communication (URLLC), and massive machine-type communications (mMTC). eMBB serves demands with high transmission rates [3]. However, B5G will present more ambitious services compared to 5G [4].

To reach these objectives, 5G/B5G networks use some key enablers such as software-defined networks (SDN), network function virtualization (NFV), cloud, and mobile edge computing (MEC). These enablers provide more flexibility to manage dynamic networks, and support more users with various QoS requirements. By leveraging virtualization, cloud-based technologies, and network softwarization, 5G/B5G can develop agile and flexible slices of networks. In this manner, network managers are able to adapt the functionality of a network based on application requirements [5].

Network slicing is a key technology of 5G/B5G that offers different network services in accordance with users' requirements by exploiting the aforementioned 5G/B5G enablers. Using network slicing, a shared physical network can be separated into several logical virtual networks referred to as slices. Each slice has its own commercial, security, traffic, and quality of service requirements, and is allocated network resources to meet the needs of its users. Each slice forms a logical

* Corresponding author.
E-mail addresses: y.azimi@urmia.ac.ir (Y. Azimi), s.yousefi@urmia.ac.ir (S. Yousefi), h.kalbkhani@uut.ac.ir (H. Kalbkhani), tkunz@sce.carleton.ca (T. Kunz).

https://doi.org/10.1016/j.comcom.2024.01.028
Received 1 August 2023; Received in revised form 8 November 2023; Accepted 25 January 2024
Available online 28 January 2024

end-to-end network from the radio access network (RAN) to the core network that provides dedicated resources (such as resource blocks (RBs), computational resources, storage, caching memory, transmission power, and network bandwidth), either through virtual networks or resource sharing/scheduling to meet the demands of its users. Due to the isolation among slices, any disturbance in one slice will not affect the functioning of another. This allows isolated slices to be managed independently of other slices, enhancing flexibility, sustainability, and reliability across the entire network [6].

Although slicing in all parts of the network is important, RAN slicing is even more important. Due to resource limitations in a RAN and the mobility of users in the network, guaranteeing the QoS of users in different slices is a major challenge. In other words, the network dynamics and the mobility of users between different RANs may increase the likelihood of service outages for the mobile users due to the lack of resources in other RANs. For this reason, to ensure the QoS for mobile users, the resources they need in the destination RANs must be reserved in such a way that the possibility of service outages is minimal. As a RAN environment is very complex and dynamic, an intelligent slicing mechanism is indispensable [7].

In recent years, a distributed architecture has been presented for RAN, which includes RRUs (radio resource units), DUs (distributed units), and CU (central unit). The RRU is responsible for processing the digital radio signals and transmits, receives, and converts the signals in the RAN. DUs are located near the RRUs and perform real-time scheduling and operations. The CU connects the 5G core network to the DUs and manages the functionality of the eNB/gNB. Additionally, it coordinates high-level protocols and access control [8]. So far, different architectures are proposed for a RAN such as distributed-RAN (D-RAN), centralized-RAN (C-RAN), open-RAN (O-RAN), and Fog-RAN (F-RAN), each of which includes different features and components. Therefore, RAN slicing algorithms need to be tailored to their requirements and architectures.

Our paper describes a distributed slicing method for resource allocation to rate-based and resource-based slices, called mobility-aware and energy-efficient federated deep reinforcement learning assisted resource allocation (ME-FDRL-RA). The proposed method allocates power and radio resources in a multi-RAN scenario including RRUs, DUs, and CU that combined in different ways for different RAN architectures by considering user mobility between RANs, inter-RANs interference, energy efficiency, and slice isolation. Also, the proposed method uses a federated averaging mechanism in order to increase convergence speed and make more accurate decisions about resource allocation to users. The proposed method uses three algorithms: federated deep reinforcement learning (FDRL), deep learning (DL), and interference-aware and energy-efficient power allocation (IA-EPA). A FDRL algorithm called federated advantage actor-critic (F-A2C) allocates resources to users on a small time-scale, while a supervised deep learning algorithm called stacked and bidirectional long-short-term memory (SBiLSTM) predicts the future allocated resources to slices on a large time-scale. In addition, to speed up the convergence of A2C algorithms, we use a federated approach to integrate the obtained knowledge by the agents in different RANs via F-A2C. To guarantee the isolation among slices in each RAN, each slice is managed independently of other slices by an F-A2C agent and an SBiLSTM model. Also, due to the frequency interference between different RANs, we use an interference-aware algorithm called IA-EPA to determine radio and power resources according to time-varying channel conditions and energy efficiency for the users of slices. Our contributions of this paper are outlined as follows:

- We consider a multi-RAN scenario by considering the inter-RANs interference between mobile and fixed users. Therefore, we propose a method called ME-FDRL-RA to distribute power and radio resources to users in each RAN slice under the mentioned scenario.
- The ME-FDRL-RA method is a distributed framework that is compatible with a variety of RAN architectures. There is a controller in each RAN, which manages the resource allocation algorithm. Also, the controllers use an orchestrator layer to exchange messages among themselves.
- The ME-FDRL-RA method uses F-A2C as a FDRL and SBiLSTM as a supervised DL to allocate resources to users and slices in each RAN. F-A2C uses other F-A2C agents' knowledge by integrating the weights of their models in the CU to speed up convergence. Also, using SBiLSTM can accurately predict the required resources for the slices in the future and prevent the reconfiguration of slices repeatedly. As a result, it increases the isolation time between the slices.
- The mobility of users can impact their QoS and the performance of other slices (such as isolation and utilization). To ensure QoS for mobile users in future RANs and to guarantee the performance of other slices, the required resources will be reserved.
- In order to solve the non-convex optimization problem of resource allocation for rate-based slices, we propose an iterative method called IA-EPA that employs the gradient descent algorithm. IA-EPA determines the required resources (power and RBs) for users in rate-based slices to optimize energy efficiency and satisfy the transmission rate of users while considering channel conditions and interference of inter-RANs. Also, IA-EPA converges fast.
- The ME-FDRL-RA method uses a distinct agent to manage each slice, which makes it easier to manage (add, remove, and stop) the slices and helps to achieve better isolation among the slices. Also, each agent is implemented independently of the other agents on a CPU thread, which means that increasing the network size (i.e., increasing the number of slices and agents) has no effect on the time and computational complexity of the proposed method. Thus, ME-FDRL-RA provides a scalable and cost-effective approach to real-life scenarios.

The paper is organized as follows. Section 2 reviews the RAN slicing approaches and summarizes the related work. In Section 3, the system model and mobility model of the proposed method are stated. The RAN slicing problem is defined and formulated as optimization problems in Section 4. We discuss in Section 5 how SBiLSTM, F-A2C, and IA-EPA can be integrated into the proposed method. Then, we provide the simulation results in Section 6 and finally, Section 7 concludes the paper.

2. Related work

In recent years, the use of intelligent techniques for RAN slicing has been explored extensively. Machine learning (ML) algorithms are one of the most important intelligent techniques, allocating resources in the RAN tailored to channel conditions and network dynamics. The advantages of online methods like reinforcement learning (RL) and online distributed learning in comparison with other ML methods for addressing the dynamic environment of RAN are discussed in [4]. To review the performance of these algorithms, we focus only on online RAN slicing proposals in this section and classify them based on how they address the challenges identified in [4], such as resource sharing, isolation among slices, mobility awareness, energy efficiency, dynamic resource allocation, slice management, and algorithmic aspects of resource allocation (e.g., convergence speed and computational and time complexity). Based on the gaps revealed in [4], we developed the proposed method to overcome them.

For dealing with the dynamic RAN environment, some proposals use a single RL agent and have high computational and time complexity. [9–14], and [15] all consider only radio resource sharing. In [9], actor-critic (AC) and LSTM are used for allocating radio resources to vehicles in a RAN. In this method, each eNB adjusts the network slice configuration using AC based on the current observations sent by vehicles and the

predicted channel condition by LSTM. The authors of [10] concentrated on different types of slices, which NGMN introduces. They use DQN to allocate resources to the slices of several MANOs in a RAN. In [11], a DQN-based method is proposed to assign the resource blocks in 5G/B5G systems while aiming to guarantee the quality of service (QoS) requirements for eMBB, uRLLC, and mMTC slices. To reduce the state–action sizes in DQN, the proposed method uses an elimination function to remove undesirable actions which cannot guarantee the QoS requirements for the desired slices. Due to the bandwidth limitation between CU and DU in a C-RAN, a DQN-based method is proposed in [12] for allocating bandwidth to each slice while satisfying the QoS of each slice. The authors in [13] introduce a solution for dynamic admission control and resource allocation in NextG radio access network slicing based on deep Q-network (DQN). The primary objective of the solution is to maximize the total weight of granted requests over time. In [14], a model-free reinforcement learning (RL) framework is used to optimize radio resource management (RRM) by adjusting control parameters in the scheduler and admission controller mechanisms for each slice. A hybrid approach of Jacobian-matrix approximation with the RL method is proposed to achieve this objective. In [15], a DQN-based method is proposed for allocating resources in F-RANs taking into account the QoS and the utilization of the resources. [16–18], and [19] perform power allocation in addition to radio resource sharing. In [16], a method called constrained discrete-continuous soft actor-critic (CDC-SAC) is presented, which aims to allocate radio resources and power to users of different slices, taking into account the queue length of packets and energy costs. [17,18] present a method to maximize the rate and real-time response to uRLLC slices compared to the eMBB slices. [17] use DQN and policy gradient AC to allocate radio and power resources, respectively. Also, in [18], a PPO method is provided for allocating sub-carriers to uRLLC and eMBB slices, with the aim to minimize the latency for uRLLC packages. [19] propose a method using RL and SDN in which the SDN controller is responsible for managing the access network to allocate radio resources to different slices and applies the RL algorithm, aiming to increase QoE of users and channel efficiency by considering the priority of each slice and the achievable data rate. In this method, transmission power is allocated based on the priority level of each slice.

To decrease computational and time complexity, some proposals use a multi-agent method based on deep reinforcement learning or transfer reinforcement learning for slicing the resources in a RAN and speeding up convergence. Radio resource sharing and algorithmic aspects of resource allocation are addressed in [20–29], and [30]. In [20], a multi-agent method called graph attention DQN (GAT-DQN) is proposed for allocating resources, taking into account the requirements of different slices and the SLA satisfaction ratio (SSR) in several BS. In this method, each BS is managed by an agent and GAT is used for coordination between BSs. Also, in [21], the authors evaluated the impact of using other RL agents on [20]. In [22], a multi-agent RL (MARL) method based on DQN is proposed for allocating radio resources to different tenants in multi-cell environments with the goal to guarantee users' SLAs and maximize resource utilization. In this method, each RL agent is assigned to a tenant, and each tenant centrally decides from observations of the environment. In [23], each slice acts as a Q-learning agent and exchanges Q values with other agents through the SDN controller to coordinate the decisions. [24,25] use a multi-branch dueling Q-network (MBDQN) for allocating radio resources by considering inter-numerology interference and maximizing network throughput. In these methods, each sub-channel is managed by an agent and the sub-actions of the agents form a general action. In [26], a distributed method based on actor-critic called EdgeSlice is proposed to guarantee the performance of each slice based on its SLA. Each slice is considered as an agent that interacts with other agents through an orchestrator. To increase the convergence speed of agents in [27,28], and [29], a combination method of transfer learning and DRL is proposed. In these methods, expert agents in a BS transfer their knowledge to other new BSs. In [28], a hierarchical and multi-agent method allocates the resources in multiple BSs of a RAN. In this method, two methods of Q-value transfer RL (QTRL) and action selection transfer RL (ASTRL) have been used to decide on resource allocation to improve the quality of current decisions using the acquired knowledge. In [29], each BS acts as a DQN agent to minimize packet latency and task processing time in uRLLC and eMBB slices. Each slice in [30] is treated as a transfer DQN agent for allocating resources to each slice in O-RAN based on their geographical features.

Some proposals have low time and computational complexities and consider radio resource sharing, isolation among slices, dynamic power allocation, energy efficiency, dynamic management of slices, and algorithmic aspects of resource allocation while ignoring the mobility of users in the resource allocation such as [31–33], and [34]. In [31], a method called LSTM-DDPG is provided for allocating radio resources in two small and large time scales in a RAN. LSTM has been used to estimate the required resources on a large time-scale. AC has been used as a well-known DDPG algorithm to allocate resources on a small time-scale. In [32], a DQN-based method using $\epsilon - greedy$ is presented to allocate radio resources and power to the C-RAN taking into account constraints of isolation and transmission rate. C-RAN includes several RRHs, and each link between RRH and users is considered as an agent. To speed up convergence in [31], the methods in [33,34] use a multi-agent and parallel method based on asynchronous advantage actor-critic (A3C). Additionally, each slice is managed by a separate agent, leading to better slice management. In these methods, A3C is used to allocate resources to users of the slice on a small time-scale, and LSTM in [33] and SBiLSTM in [34] are used to allocate resources to slices on a large time-scale for ensuring the isolation among slices. In addition, [34] uses an iterative algorithm to determine the required resources of users in slices by considering the slice requirements.

Some proposals use a dynamic power allocation and resource sharing method by considering the algorithmic aspects of the proposed method such as low time and computational complexities and convergence speed, e.g., [35,36], and [37]. [35] presents a federated learning model called FL-CA that uses actor-critic (AC) for radio and power allocation. The Deep Federated Q-Learning (DFQL) is proposed in [36] for the allocation of radio and power resources for virtual slices in industrial IoT applications using NFV and SDN. Also, a multi-agent strategy based on proximal policy optimization is suggested in [37] to improve management in end-to-end slices and convergence speed.

In summary, using online reinforcement learning methods can be a good option for dealing with a dynamic RAN environment while considering the aforementioned identified challenges. Although many methods are based on reinforcement learning, most of them do not take into account the all identified challenges and are not suitable for real-life scenarios. In general, the superiority of the proposed method compared to [9–19] is to address isolation among slices, mobility awareness, energy efficiency, slice management, and algorithmic aspects of resource allocation. Also, the proposed method considers isolation among slices, mobility awareness, energy efficiency, and slice management compared to [20–30]. Compared to [31–34], the proposed method addresses mobility awareness, dynamic power allocation and energy efficiency by considering the inter-RANs interference, and algorithmic aspects of resource allocation besides the isolation among slices and slice management. In addition, the proposed method takes into account isolation among slices, mobility awareness, and energy efficiency compared to [35–37]. To the best of our knowledge, the effect of user mobility until now has not been considered on isolation among slices in a multi-RAN scenario. Therefore, we consider a multi-RAN scenario and propose a multi-agent and distributed method for dynamically allocating power and radio resources to users of slices, which addresses isolation among slices, mobility awareness, energy efficiency with regard to the inter-RANs interference, slice management, and algorithmic aspects of resource allocation such as convergence speed and computational and time complexity.

3. System model

Fig. 1 describes a system that has multiple RANs, where B denotes the number of RANs. Slices are layered on top of a shared physical infrastructure in each RAN. It is assumed that the RANs have an identical number of slices and computational resources. We consider a distributed architecture for each RAN according to the O-RAN architecture including RRUs, DUs, and CU, in which DUs and CU are centralized in a baseband unit (BBU). The O-RAN architecture's radio management has two layers — the near real-time RIC (Radio Intelligent Controller) and the non-real-time RIC, which are located in the BBU part. The near real-time RIC handles RAN management on a small time scale, while the non-real-time RIC configures policies and performance in each slice on a larger time scale, and employs techniques such as machine learning for traffic control in offline mode [38]. Each RAN is managed by a controller in DU, and the DU controller is responsible for running the resource allocation algorithm in its own RAN. A DU controller communicates with other DU controllers via an orchestrator layer. Each RAN is composed of a number of RRUs which are positioned in a given geographical area, and provides a pool of power and RBs resources shared by all RANs in that region. Also, CU runs an aggregator for using the obtained knowledge from the agents to increase the convergence speed in our FDRL methods.

This paper assumes that perfect channel state information (CSI) can be obtained in the DU controllers [34,39]. Also, each DU controller sends its CSI to other DU controllers if its CSI is changed.

We denote by M^b the set of slices in RAN b. That set is divided into resource-based slices $M_I^b \subset M^b$ and rate-based slices $M_{II}^b \subset M^b$ [33,34]. Each slice has a different service level agreement (SLA). Also, let U_m^b correspond to users of the slice m in RAN b. In resource-based slices, we always allocate a previously specified and fixed amount of resources (RBs and power) to their users. The required resources in rate-based slices can vary with regard to the transmission rate and channel condition. Indeed, each RAN allocates resources to the users of the rate-based slices to ensure a minimum transmission rate, maximize energy efficiency, while taking into consideration channel conditions. We allocate two types of resources in each RAN: transmission power and RBs. The allocated resources are used exclusively by users of each slice in each RAN until they are released. Each slice in the RAN comprises a sharing account book to exchange and coordinate important information (e.g., states of the system and slices) among slices. In this paper, two types of users have been considered: (i) fixed users who have a permanent location in the network area, and (ii) mobile users who use a mobility model and change their location. Furthermore, the users' requests in arrival rate in slice m in RAN b is $\lambda[m,b]$, and the users' requests departure rate in slice m in RAN b is $\mu[m,b]$. We represent the specification of slice m, $m \in M^b$ in RAN b with a three-tuple $\{r_{m,b}^{Total}(t), h_{m,b}^{Total}(t), T_{m,b}^{th}\}$ at time t, where $r_{m,b}^{Total}(t)$ denotes the total allocated amount of RBs for mobile and fixed users of slices, $h_{m,b}^{Total}(t)$ denotes the total required amount of RBs for mobile and fixed users of slices, and $T_{m,b}^{th}$ denotes the minimum required duration for ensuring isolation.

According to [38], we use two layers for resource allocation: the slices layer and the users layer. For the slices layer, we use SBiLSTM, and for the users layer, we use FDRL. We first use the SBiLSTM on a large time scale, i.e., in a prediction window (PW), for resource allocation to each slice. Then, we use FDRL on a small time scale, i.e., each time step, for resource allocation to users in the slices. The term prediction window (PW) refers to a large time-scale made up of numerous short time-steps. At each time-step, resources are allocated to users of the slices on a small time-scale, and at the beginning of each PW, resources are allocated to slices on a large time-scale. After allocating resources to each slice, we maintain a pool of unallocated resources, which we call "residual resources". When the requests of slice m in RAN b come into the system based on arrival rates $\lambda[m,b]$, the FDRL method will decide on the resource allocation to the users.

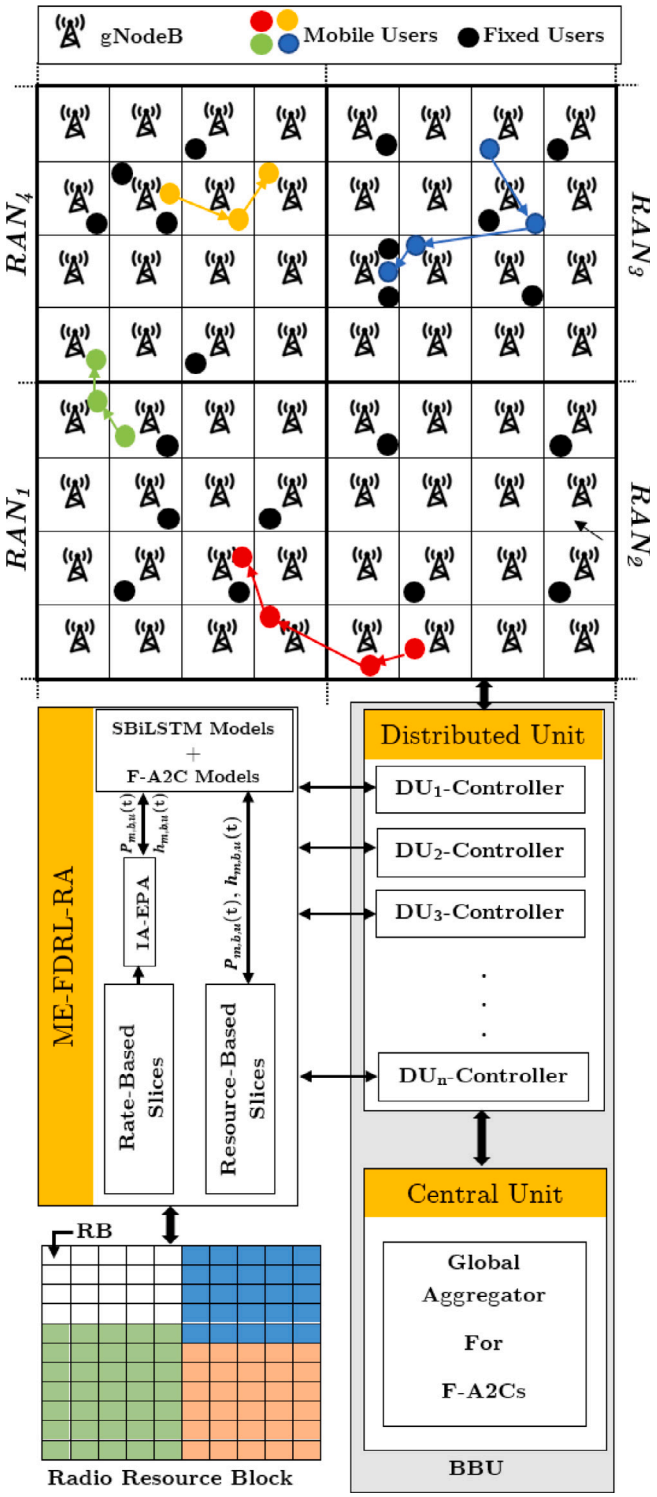


Fig. 1. System model.

Earlier, we mentioned that the slice type (that is, resource-based or rate-based) determines how much resources are required for its users. The number of RBs for resource-based slices is always fixed and they use the maximum transmission power. Hence, the corresponding users receive the maximum available power that decreases the effect of inter-RAN interference. According to channel conditions and transmission rate, required power and RBs for users in rate-based slices are determined by the IA-EPA method. Due to the interference between RANs,

Table 1
Notations and symbol definitions.

Symbol	Description
B	The number of RANs
Θ_b	The total of all RBs in RAN b
$P_{total,b}$	The total of power resources in RAN b
M^b	The set of slices in RAN b
M^b_I	The set of rate-based slices in RAN b
M^b_{II}	The set of resource-based slices in RAN b
$U^b_m, m \in M^b$	The set of users belong to the slice m in RAN b
T_Δ	Prediction Window (PW)
$D_{m,b}$	Difference between the amount of resources before and after reconfiguration for slice m in RAN b
$P_{m,b,u}(t)$	Amount of required power for user u in slice m of RAN b at time t
$N_{m,b,u}(t)$	Counter for counting amount of required RBs for user u in slice m of RAN b at time t
$r^{Total}_{m,b}(t)$	The total amount of allocated RBs for slice m of RAN b at time t
$r_{m,b,u}(t)$	Amount of allocated RBs for user u in slice m of RAN b at time t
$h^{Total}_{m,b}(t)$	The total amount of required RBs for slice m of RAN b at time t
$h_{m,b,u}(t)$	Amount of required RBs for user u in slice m of RAN b at time t
$N_{\Theta_{x,b}}(t)$	The number of residual RBs in RAN b
η_{SE}	Spectrum efficiency
η_{EE}	Energy efficiency
$\lambda[m,b],\ \mu[m,b]$	Arrival and departure rates of users' requests in slice m of RAN b
$T^{th}_{m,b}$	The minimum threshold of the length of duration to ensure isolation for slice m in RAN b

considering a fixed value for transmission power will decrease the QoS of the users. In addition, using the maximum transmission power for all users will increase energy consumption, the level of inter-RAN interference, and the operational costs of the service provider. To meet QoS requirements for users, optimum power levels are determined by evaluating energy efficiency over time-varying channels. In Fig. 1, $P_{m,b,u}(t)$ and $h_{m,b,u}(t)$ are the amount of required power and RBs for user u of slice m in RAN b. Overall, the combination of SBiLSTM, F-A2C, and IA-EPA is suggested for RAN resource slicing. Table 1 summarizes the notations and symbols used throughout the paper.

As illustrated in Fig. 1, we have fixed and mobile users. The position of the fixed users is constant during the simulated time period, but the movements of the mobile users obey a mobility model. In this paper, we use a mobility model called small world in motion (SWIM) [40] to generate small worlds. Using the mobility model in simulations is easy thanks to its simplicity, and the mobility pattern of the nodes is based on a simple intuition of human mobility. In fact, people are less inclined to go to places far away from home where they meet more other people. Applying this simple rule by SWIM can imitate the social behavior of people in real life for the mobility of nodes.

In SWIM, a point called home is considered for each node, which is selected randomly and uniformly over the network. Then, each node must calculate a weight for the possible destinations. The weight increases with the popularity of the place and decreases with distance from home. The popularity of a place is calculated by a user, which equals the ratio of users the user sees in that place to the total number of users. Indeed, the weight indicates the probability of choosing that place as the next destination of the node. After selecting the destination, the node moves towards it in a straight line and at a constant speed. When the node reaches the destination, it pauses for a period of pause time, selects the next destination, and continues the movement. We consider the network area as a grid with equal cell sizes, and RRUs are located at the cells' center. Also, each RRU can cover the entire cell.

Suppose A is a mobile node and h_A is its home, and cell C is a possible destination for A. The weight of cell C can be calculated as follows:

$$W(C) = \alpha \times Distance(h_A, C) + (1-\alpha) \times Seen(C) \tag{1}$$

where α is a constant between 0 and 1. For a small amount of α, the node inclines to go to popular places and for a large amount of α, the node will be more inclined to visit places near its home. $Seen(C)$ denotes the number of nodes that node A met in cell C the last time it reached cell C. So initially, it would be zero. It is updated each time node A reaches a destination in cell C. $Distance(h_A, C)$ is a function that decays with increasing distance between node A and cell C. The distance function can be expressed as follows:

$$Distance(x, C) = \frac{1}{(1 + k\|x - C\|)^2} \tag{2}$$

where k is a scaling factor between 0 and 1. We consider the center of cell C to calculate the distance between cell C and a given point like x, which is randomly selected as a final destination in the destination cell.

4. Problem definition and formulation of resource allocation

To effectively conceive the resource allocation problem, we divide the problem into long-term and short-term subproblems, which are executed on different time scales. The long-term subproblem is executed in each prediction window (PW) as a large time-scale which is built by multiple time steps. Whereas, the short-term subproblem is executed in each time step as a small time-scale. The long-term subproblem is designed to determine the required resources for the slices on the next PW. In contrast, the short-term subproblem is deployed as the resource management algorithm in a RAN, which allocates the allocated resources of the slice to its users:

Problem 1 (*Resource Slicing with Mobility Awareness on a Large Time-Scale*)**.** Each slice in a RAN must allocate resources on a large scale (for the next PW) by taking into account mobile and fixed users' traffic to predict the required resources for the next PW and meet QoS requirements. Therefore, the error of the predicted amount for the total allocated resources including the traffic of the mobile and fixed users must be minimal compared to the actual total allocated resources [34]. For this reason, we minimize the mean-square-error (MSE) value, which is calculated using the actual value $r^{Total}_{m,b}(t)$ and the predicted value $\hat{r}^{Total}_{m,b}(t)$ of the allocated resources to slice m in RAN b. Hence, on a large time-scale, the resource allocation problem can be expressed as follows:

$$\arg\min_{\hat{r}^{Total}_{m,b}(t)} \quad \sum_{b=1}^{B}\sum_{m=1}^{M^b}\frac{1}{T_\triangle}\sum_{t=1}^{T_\triangle}\left|r^{Total}_{m,b}(t) - \hat{r}^{Total}_{m,b}(t)\right|^2 \tag{3a}$$

$$s.t. \quad \sum_{b=1}^{B}\sum_{m=1}^{M^b}\hat{r}^{Total}_{m,b}(t) \le \Theta_b \tag{3b}$$

where Θ_b denotes the allocable RBs in RAN b. Accordingly, two scenarios may occur for resource allocation deriving from Eq. (3): (a) Whenever actual resource ($r^{Total}_{m,b}(t)$) usage exceeds the predicted amount of resources ($\hat{r}^{Total}_{m,b}(t)$), i.e., $r^{Total}_{m,b}(t) > \hat{r}^{Total}_{m,b}(t)$, slice m may need to ask

for more resources from the residual resource. (b) If $r_{m,b}^{Total}(t) \leq \hat{r}_{m,b}^{Total}(t)$, then the allocated resources remain unchanged (till the allocation likely changes in the next PW) to ensure slice isolation.

Problem 2 (*Resource Allocation to Users Per Slice on a Small Time-Scale*)**.** We introduce online resource allocation on a small time-scale to control unpredictable traffic changes on a large time-scale [34]. It decides whether to admit users in each slice. This optimization problem allocates a minimum amount of required resources to users per slice with the aim of guaranteeing isolation per slice. Because resources are frequently requested per slice, resource reconfiguration increases system overhead and reduces slice performance. As a result, it is imperative to maintain the isolation degree at an appropriate level. In order to achieve this, assume that $D_{m,b}(t) = |r_{m,b}^{Total}(t) - r_{m,b}^{Total}(t-\tau)|$ is the difference between the amount of the allocated resources before and after the resource reconfiguration per slice m in RAN b during $t-\tau$ and t, and $\Omega_{m,b}(D_{m,b}(t)=0)$ counts the consecutive time steps in which $r_{m,b}^{Total}(t)$ has been unchanged. Therefore, the small time-scale problem can be formulated like this:

$$\min \quad \sum_{b=1}^{B}\sum_{m=1}^{M^b}\sum_{u=1}^{U_m^b} r_{m,b,u}(t) \tag{4a}$$

$$s.t. \quad \sum_{b=1}^{B}\sum_{m=1}^{M^b}\sum_{u=1}^{U_m^b} r_{m,b,u}(t) \leq \Theta_b \tag{4b}$$

$$\sum_{b=1}^{B}\sum_{m=1}^{M^b}\sum_{u=1}^{U_m^b} r_{m,b,u}(t) \geq h_{m,b,u}(t) \tag{4c}$$

$$\sum_{b=1}^{B}\sum_{m=1}^{M^b} \Omega_{m,b}(D_{m,b}(t)=0) \geq T_{m,b}^{th} \tag{4d}$$

where $T_{m,b}^{th}$ is the minimum duration to ensure isolation for slice m in RAN b, $h_{m,b,u}(t)$ denotes the amount of required RBs for user u in slice m of RAN b at time t, and $r_{m,b,u}(t)$ denotes the amount of allocated RBs to the accepted user u in slice m of RAN b at time t. In rate-based slices, the user's $h_{m,b,u}(t)$ is determined by the Shannon-Hartley theorem based on the signal-to-interference-noise ratio (SINR), the distance from the base station, as well as the required transmission rate $R_{m,b}^{th}$ [34]. Thus, the user's $h_{m,b,u}(t)$ for rate-based slices in RAN b at time step t would be as follows:

$$\begin{aligned} h_{m,b,u}(t) &= \frac{R_{m,b}^{th}}{wrb \times log_2(1+\frac{P_{m,b,u}(t)\times G_{m,b,u}^2(t)}{(N_0+N_{int})_{m,b,u}(t)})} \\ &\geqslant \frac{R_{m,b}^{th}}{wrb \times log_2(1+\frac{P_{max}\times G_{m,b,u}^2(t)}{(N_0+N_{int})_{m,b,u}(t)})} \end{aligned} \tag{5}$$

where $G_{m,b,u}(t)$ is the time-varying Rayleigh fading channel gain of the transmission, which is affected by path loss $PL_{m,b,u}(t)$ and Rayleigh fading $f_{m,b,u}(t)$, and equals $G_{m,b,u}(t) = f_{m,b,u}(t)/PL_{m,b,u}(t)$ [34]. The path loss of the link between transmitter and receiver equals $PL_{m,b,u}(t) = 10 * n * log_{10}(d) + C$, where n is the path loss exponent, d is the distance between UE and RRU in meter, and C denotes a constant. $P_{m,b,u}(t)$ is the down-link transmission power between UE and RRU, N_0 is the variance of Additive White Gaussian Noise (AWGN) of channel, N_{int} denotes the maximum interference from other RANs on the selected RBs to assign to user u. The interference noise on an RB in a RAN equals the sum of the created noise from other RANs on the selected RB. Also, wrb represented the bandwidth of the RB. Furthermore, the inequality in Eq. (5) shows that if the RRU transmits with maximum power, the minimum number of RBs is required [34].

Problem 3 (*Determining the Required Resources for Users of Slices*)**.** In our scenario, a fixed amount of the power $P_{m,b,u}(t)$ and a fixed number of required RBs $h_{m,b,u}(t)$ are assigned to resource-based slices. Due to the inter-RAN interference, the users' devices in the resource-based slices use the maximum power P_{max} to overcome interference noise by the other RANs. It will work in all cases if it works for the worst case scenario. The purpose of this section is to determine the user's resources in a rate-based slice. This problem seeks to maximize energy efficiency taking into consideration the user's transmission rate, transmission power, and RB constraints. To increase energy efficiency, and spectrum efficiency, and improve the QoS of users, the resources of the rate-based slices should be allocated with regard to channel conditions such as inter-RAN interference and user mobility. For this reason, the resources should be updated considering changes in channel conditions over time. The resources determination problem is expressed as follows:

$$\begin{aligned} \underset{N_{m,b,u}(t),P_{m,b,u}(t)}{\arg\max} \quad (\eta_{EE} &= \frac{\eta_{SE}}{P'_{total,b}} \\ &= \sum_{b=1}^{B}\sum_{m=1}^{M_{II}^b}\sum_{u=1}^{U_m^b} \frac{log_2(1+P_{m,b,u}(t)\times X_{m,b,u}(t))}{P_{circuit}+P_{m,b,u}(t)}) \end{aligned} \tag{6a}$$

$$s.t \quad N_{m,b,u}(t)\times wrb \times log_2(1+P_{m,b,u}(t)\times X_{m,b,u}(t)) \geqslant R_{m,b}^{th} \tag{6b}$$

$$0 \leqslant P_{m,b,u}(t) \leqslant P_{max} \tag{6c}$$

$$1 \leqslant N_{m,b,u}(t) \leqslant N_{r_{m,b}}(t) + N_{\Theta_{s,b}}(t) \tag{6d}$$

where $X_{m,b,u}(t)$ equals $G_{m,b,u}^2(t)/(N_0+N_{int})_{m,b,u}(t)$, $N_{m,b,u}(t)$ represents the minimum number of idle RBs that can assign to user u in the slice m of RAN b, $N_{r_{m,b}}(t)$ denotes the number of remaining allocated RBs for the slice m in RAN b, $N_{\Theta_{s,b}}(t)$ are the number of residual RBs in RAN b and $P_{circuit}$ is the circuit power. Optimal energy efficiency is achieved when the minimum number of RBs ($N_{m,b,u}(t)$) and transmission power ($P_{m,b,u}(t)$) satisfy the problem's object [34]. As a result, the required RBs ($h_{m,b,u}(t)$) for user u in the slice m of RAN b equals $N_{m,b,u}(t)$.

Our resource allocation problem, which includes Problems 1, 2, and 3, is classified as one of the NP-hard problems [34]. So, the time and computational complexities of these problems preclude us from solving them using static optimization techniques in real-time. Therefore, we use SBiLSTM and F-A2C for addressing Problems 1 and 2, respectively. Also, a gradient descent method based on the Lagrange multiplier called IA-EPA is used to solve Problem 3. Generally, we propose the ME-FDRL-RA method (including SBiLSTM, F-A2C, and IA-EPA) to solve Problems 1, 2, and 3. In the following, we explain this ME-FDRL-RA method in detail.

5. Proposed solutions

As a solution to the problems in Section 4, a distributed method named ME-FDRL-RA is developed. The ME-FDRL-RA components are introduced as follows (Fig. 2):

- **DU-Controllers:** Each RAN is managed by a DU-Controller, and the DU-Controller runs DRL, SBiLSTM, and IA-EPA algorithms to assign power and radio resources to the users and slices.
- **Orchestrator:** We use an orchestration layer to exchange information (e.g., CSI and user mobility) and coordinate between the controllers in DU.
- **Parallel F-A2Cs for the set of slices per RAN:** In the ME-FDRL-RA algorithm, each slice in a RAN represents an F-A2C agent that allocates resources to its users independently of other agents in the RAN.
- **Shared account book for each RAN:** Each slice in a RAN shares its own resource allocation information among slices of the RAN via the shared account book. Information logged about slices during the current PW and previous PWs assists in estimating the allocated resources for each slice in the next PW.

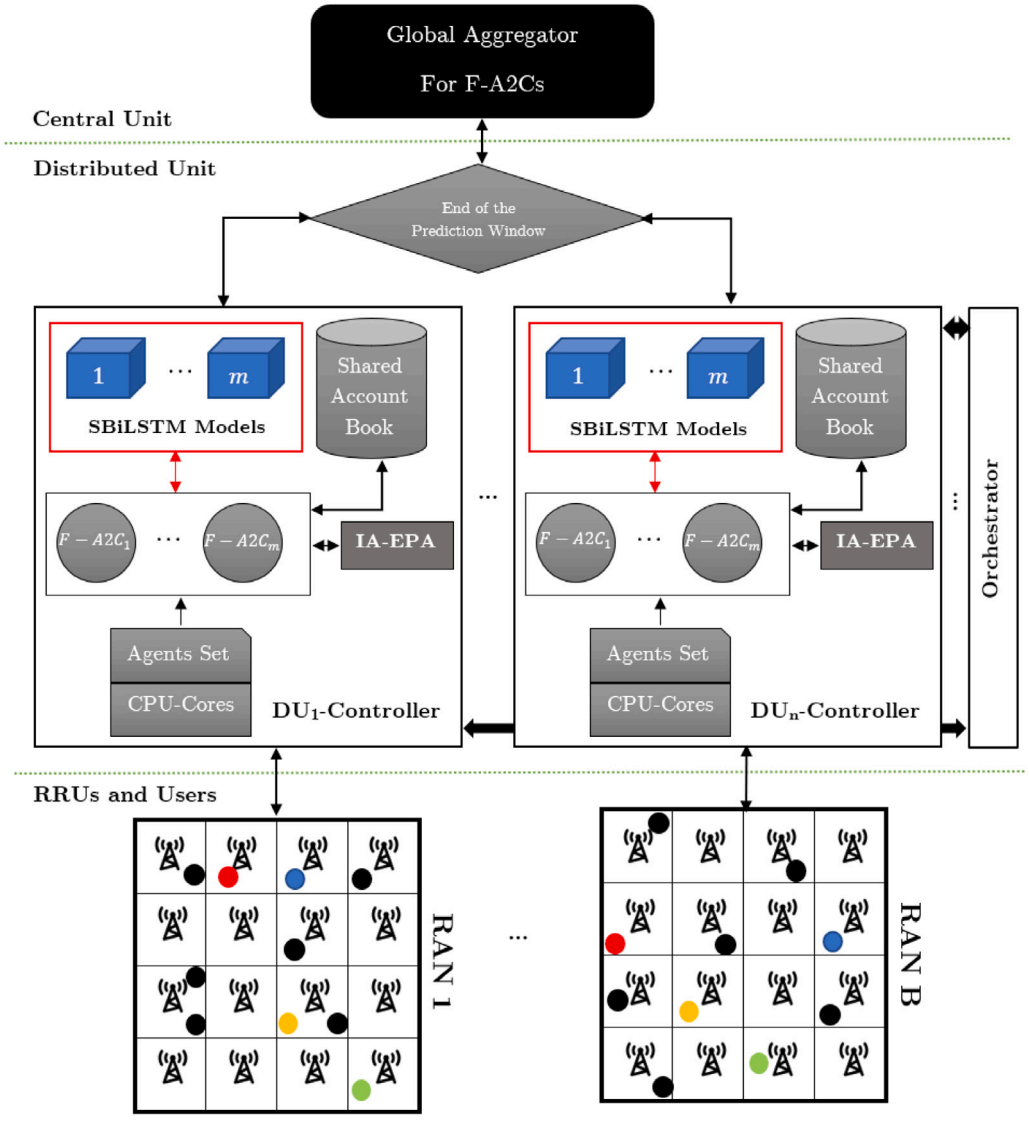


Fig. 2. ME-FDRL-RA components.

- **IA-EPA method:** IA-EPA is a gradient descent method based on Lagrange multipliers that calculates the required resources for users in rate-based slices. It is an iterative method that finds the optimal answer in a few iterations by considering the channel condition and energy efficiency. As mentioned in Section 3, the required resources for users of resource-based slices are fixed and pre-specified.
- **SBiLSTM model for each slice in each RAN:** To guarantee the isolation among slices, we use the SBiLSTM models to predict the required resources to the slices of each RAN in the next PW.
- **Global Aggregator for F-A2Cs :** To increase the convergence speed and improve the decisions in each F-A2C agent, we calculate the average of the agents' weights in the global aggregator in the CU at the end of each PW. Then, we share them with the agents.

5.1. Large time-scale prediction

Based on O-RAN architecture [38], the location of users is trackable and can be accessible from BBUs. Since mobile users are moving between different RANs, the number of users in each RAN changes dynamically and this degrades the resource allocation performance. To address this issue, we track the total traffic of fixed and mobile users who move from one RAN to another in each time step and use this data as input for stacked and bidirectional Long-Short-Term-Memory (SBiLSTM).

In Problem 1 in Section 4, we want to find the predicted amount of $\bar{r}^{Total}_{m,b}(t)$ in the next PW for slice m in RAN b so that the minimum value of MSE is obtained for the difference between actual allocated amount of $r^{Total}_{m,b}(t)$ and predicted amount of $\bar{r}^{Total}_{m,b}(t)$. To this end, we use a supervised DL method, i.e., SBiLSTM to predict the allocated resources to the slices in the next PW on a large time-scale using the obtained information of the required resources in the previous PWs.

To predict the allocated resource of slice m in RAN b, we use $h^{Total}_{m,b}(t)$ as the SBiLSTM input data for model m in RAN b, which are collected at each time step from the previous and current PWs. $h^{Total}_{m,b}(t)$ is the sum of the required resources for fixed users $h^{Fixed}_{m,b}(t)$ of slice m in RAN b and the required resource for mobile users $h^{Mobile}_{m,b}(t)$ of slice m in RAN b.

Let $D_{input,m,b} = \{h^{Total}_{m,b}(t-2PW), \ldots, h^{Total}_{m,b}(t-PW), \ldots, h^{Total}_{m,b}(t)\}$ be the input data for the prediction of the allocated resources in slice m of RAN b in the next PW. Assume $Y_{predicted,m,b} = \{y_{m,b}(t+1), \ldots, y_{m,b}(t+PW)\}$ be the predicted values for the required resources of the slice m in RAN b in the next PW. To determine the allocated resources to

slice m in RAN b, we calculate a confidence interval from the predicted values according to $\hat{r}^{Total}_{m,b}(t) \in \bar{Y}_{predicted,m,b} \pm z_{\frac{(1-\chi)}{2}} \cdot \frac{\sigma(Y_{predicted,m,b})}{\sqrt{PW}}, t \in [t, t+T_{\triangle}]$. To this end, we need the sample mean ($\bar{Y}_{predicted,m,b}$) and standard deviation $\sigma(Y_{predicted,m,b})$ of the predicted values [33]. Thus, the allocated resources to slice m in RAN b fall within the obtained interval, where χ is a confidence level. χ is between 0 and 1 and can be different for each slice and dynamically tuned during the simulation. To guarantee the QoS of users, improve the performance of the slice, and overcome the possible errors in the predicted values, we assign the upper bound value of the confidence interval to $\hat{r}^{Total}_{m,b}(t)$ for slice m in the next PW. Probably worth mentioning though that [33,34] did not consider mobility, which would justify evaluating how well this works under mobility.

5.2. Interference Aware and Energy-efficient Power Allocation (IA-EPA)

Since we considered the multi-RAN scenario and reusing the available RBs between them, there is interference from one RAN to other RANs. This interference degrades the down-link of the considered RAN and reduces the SINR. Hence, we should consider the interference in our analysis to ensure the QoS. In Problem 3 in Section 4, we want to find the minimum required resources for users of rate-based slices on a small time-scale while considering the energy efficiency and channel condition such as inter-RAN frequency interference subject to constraints (6b), (6c), and (6d). Obtaining the optimal solution for Problem 3 incurs high time and computational complexity. Hence, we propose a gradient-descent-based sub-optimal method called IA-EPA with a low time and computational complexity. The IA-EPA algorithm improves resource allocation efficiency by achieving a trade-off between the achievable rate and power consumption while considering the interference caused by other transmitters. It can adapt to dynamic changes in the network environment and user demands by updating the power allocation accordingly. Overall, the IA-EPA method has several key features as follows:

- The algorithm is distributed, which means that each transmitter only requires its own channel state information and interference level, and does not need to coordinate with other transmitters or a central controller.
- The algorithm exploits the properties of the standard interference function, which characterizes the relationship between the signal-to-interference-plus-noise ratio (SINR) and the power allocation vector. The standard interference function satisfies some conditions, such as monotonicity, scalability, and boundedness, that make the gradient descent method converge to a unique fixed point.
- The algorithm convergence to an optimal solution using the gradient descent method. In our particular framework, we have shown in Section 6 that the solution is the same as the global optimum solution obtained by Lingo.

The IA-EPA method is summarized in Algorithm 1. The IA-EPA method decomposes the joint optimization problem (6) into two sub-problem [34]:

1. The first sub-problem determines the minimum number of idle RBs ($N_{m,b,u}$) that the RAN can assign to the user.
2. The second sub-problem determines the minimum amount of $P_{m,b,u}(t)$ based on the considered RBs in sub-problem 1 to maximize energy efficiency.

The idle RBs for the first sub-problem can be selected from $N_{m,b,u}(t) \leqslant N_{r_{m,b}}(t) + N_{\Theta_{s,b}}(t)$, where $N_{r_{m,b}}(t)$ are the idle RBs in Slice m and $N_{\Theta_{s,b}}(t)$ are the residual RBs that have not allocated to the slices in RAN b.

To solve (6), we must show that the following problem has an optimal solution based on the considered $N_{m,b,u}$ in sub-problem 1 (Lines 3–10):

$$\underset{P_{m,b,u}(t)}{\arg\max} \quad (\eta_{EE} = \frac{\eta_{SE}}{P'_{total}} = \frac{log_2(1 + P_{m,b,u}(t) \times X_{m,b,u}(t))}{P_{circuit} + P_{m,b,u}(t)}) \tag{7a}$$

$$s.t \quad N_{m,b,u}(t) \times wrb \times log_2(1 + P_{m,b,u}(t) \times X_{m,b,u}(t)) \geqslant R^{th}_{m,b} \tag{7b}$$

$$0 \leqslant P_{m,b,u}(t) \leqslant P_{max} \tag{7c}$$

The EE optimization problem in (7) is non-convex, which is arduous and hard to solve [34]. Applying non-linear fractional programming, we transform the non-convex optimization problem into a concave problem:

$$q^*_s(t) = \ max \quad \frac{\eta_{SE}}{P_{circuit} + P_{m,b,u}(t)} = \frac{\eta_{SE}(P^*_{m,b,u}(t))}{P_{circuit} + P^*_{m,b,u}(t)} \tag{8}$$

Considering (8), we can transform the problem in (7) into subtraction form as follows:

$$P^*_{m,b,u}(t) = \underset{P_{m,b,u}(t)}{\text{argmax}} \quad (\eta_{SE} - q_s(t)(P_{circuit} + P_{m,b,u}(t))) \tag{9a}$$

$$s.t \quad N_{m,b,u}(t) \times wrb \times log_2(1 + P_{m,b,u}(t) \times X_{m,b,u}(t)) \geqslant R^{th}_{m,b} \tag{9b}$$

$$0 \leqslant P_{m,b,u}(t) \leqslant P_{max} \tag{9c}$$

We apply the solution used in [34] to solve (9) (Lines 11–29). We obtain the optimum value of $P^*_{m,b,u}(t)$ in (10) as follows (Line 16):

$$P^*_{m,b,u}(t) = \left[\frac{N_{m,b,u}(t) \times wrb \times (1 + \lambda_1(t))}{ln(2)(\lambda_3(t) + q_s(t) - \lambda_2(t))} - \frac{1}{X_{m,b,u}(t)} \right]^+ \tag{10}$$

where $[x]^+ \triangleq max\{0, x\}$. Also, $\lambda_1(t)$, $\lambda_2(t)$ and $\lambda_3(t)$ are the Lagrange multipliers. In addition, $q_s(t)$ is a multiplier generated since the problem is transformed into a concave problem. We can find the Lagrange multipliers via gradient method and update the multipliers in the following manner: (Lines 19–21):

$$\lambda^{i+1}_1(t) = \Big[\lambda^i_1(t) - \mu^i_{\lambda_1}(t)(N_{m,b,u}(t) \times wrb \times log_2(1 + \hat{P}_{m,b,u}(t) X_{m,b,u}(t)) - R^{th}_{m,b}) \Big]^+ \tag{11a}$$

$$\lambda^{i+1}_2(t) = \left[\lambda^i_2(t) - \mu^i_{\lambda_2}(t) \hat{P}_{m,b,u}(t) \right]^+ \tag{11b}$$

$$\lambda^{i+1}_3(t) = \left[\lambda^i_3(t) - \mu^i_{\lambda_3}(t)(\hat{P}_{m,b,u}(t) - P_{max}) \right]^+ \tag{11c}$$

where i is the iteration counter and $\mu^i_{\lambda_1}(t)$, $\mu^i_{\lambda_2}(t)$ and $\mu^i_{\lambda_3}(t)$ are positive step sizes in iteration i to achieve an optimum value. Choosing the step sizes is a trade-off between optimality and convergence speed, so, we have used $\mu^{i+1}(t) = \mu^i(t)/l$ as step size (Lines 22 and 23). The algorithm ends when condition $(\eta_{SE} - q_s(t)(P_{circuit} + P_{m,b,u}(t)) \leqslant \varepsilon)$ is satisfied (ε is the maximum tolerance), or it reaches the maximum number of iterations (i.e., I_{max}). The optimum values are obtained when the condition on Line 24 is satisfied.

5.3. Small time-scale prediction

In Problem 2 in Section 4, we want to allocate the minimum $r_{m,b,u}(t)$ on a small time-scale for user u in slice m of RAN b with regard to constraints (4b), (4c) and (4d). We first model the resource allocation problem on a small time-scale for each slice in a RAN as a Markov Decision Process (MDP) and then solve it using the federated advantage actor-critic (F-A2C) algorithm. The F-A2C algorithm is formed of an actor, a critic and a federated aggregator, in which the actor interacts with the environment. First, the actor chooses the best action conforming to the current policy and receives a reward. Then, the critic updates the state-value function according to the temporal difference (TD) error.

Algorithm 1: Pseudocode of Interference Aware and Energy-Efficient Power Allocation (IA-EPA) Method For Users in Rate-Based Slices

Input: $R^{th}_{m,b}$, wrb, $P_{Total,b}$, and P_{max}
Output: The selected RBs, P, Q

1 Initialize $R^{th}_{m,b}$, wrb
2 Initialize $P_{max} \longleftarrow \min\{P_{max}, P_{Total,b}(t)\}$
3 **while** $P_{m,b,u} > P_{max} or P_{m,b,u} < 0$ **do**
4 $N_{m,b,u} \longleftarrow$ Select RBs from idle RBs
5 Calculate $G_{m,b,u}(t) = f_{m,b,u}(t)/PL_{m,b,u}(t)$
6 Calculate $X_{m,b,u}(t) \longleftarrow G^2_{m,b,u}(t)/(N_0 + N_{int})_{m,b,u}(t)$
7 $P_{m,b,u}, Q_{m,b,u} = GD\_M\ (N_{m,b,u}, wrb, R^{th}_{m,b}, X_{m,b,u}(t), P_{max})$
8 **if** $N_{m,b,u}$ *Not Found* **then**
9 **return** Null
10 **return** $N_{m,b,u}, P_{m,b,u}, Q_{m,b,u}$
11 **Function** GD_M$(N_{m,b,u}, wrb, R^{th}_{m,b}, X_{m,b,u}(t), P_{max})$:
12 Initialize $q_s(t)$, ε, I_{max} and L_{max}
13 **for** $i = 1$ *to* I_{max} **do**
14 Initialize $\lambda_1(t)$, $\lambda_2(t)$ and $\lambda_3(t)$
15 **for** $l = 1$ *to* L_{max} **do**
16 $\hat{P}^l_{m,b,u}(t) \longleftarrow \left[\frac{(N_{m,b,u}\times wrb)(1+\lambda^l_1(t))}{ln(2)(\lambda^l_3(t)+q_s(i)-\lambda^l_2(t))} - \frac{1}{X_{m,b,u}(t)}\right]$
17 **if** $\hat{P}^l_{m,b,u}(t) < 0$ **then**
18 **return** $\hat{P}^l_{m,b,u}(t), 0$
19 $\lambda^{l+1}_1(t) \longleftarrow \left[\lambda^l_1(t) - \mu^l_{\lambda_1}(t)(N_{m,b,u} \times wrb \times log_2(1 + \hat{P}^l_{m,b,u}(t)X_{m,b,u}(t)) - R^{th}_{m,b})\right]$
20 $\lambda^{l+1}_2(t) \longleftarrow \left[\lambda^l_2(t) - \mu^l_{\lambda_2}(t)\hat{P}^l_{m,b,u}(t)\right]$
21 $\lambda^{l+1}_3(t) \longleftarrow \left[\lambda^l_3(t) - \mu^l_{\lambda_3}(t)(\hat{P}^l_{m,b,u}(t) - P_{max})\right]$
22 $\mu^{l+1}_{\lambda_2}(t) = \mu^l_{\lambda_2}(t)/l$
23 $\mu^{l+1}_{\lambda_3}(t) = \mu^l_{\lambda_3}(t)/l$
24 **if** $(\eta_{SE} - q_s(P_{circuit} + P^l_{m,b,u}(t)) \leqslant \varepsilon)$ **then**
25 $P^*_{m,b,u}(t) \longleftarrow \hat{P}^l_{m,b,u}(t)$
26 $q^*_s(t) \longleftarrow \eta_{SE}(P^*_{m,b,u}(t))/P_{circuit} + P^*_{m,b,u}(t)$
27 **return** $P^*_{m,b,u}, q^*_s$
28 **else**
29 $q^{i+1}_s(t) \longleftarrow \eta_{SE}(\hat{P}^l_{m,b,u}(t))/P_{circuit} + \hat{P}^l_{m,b,u}(t)$

After that, the actor can update its policy. In order to improve the agent's decision-making and convergence speed, periodic updates to the actor and critic networks are performed by a federated aggregator.

The problem for slice m in RAN b can be modeled as an MDP model with a five-tuple $\{S_{m,b}, A_{m,b}, R_{m,b}, \Pi_{m,b}, \gamma_{m,b}\}$ where $S_{m,b}$ is the state space to demonstrate the system environment, $A_{m,b}$ is the action space, to measure the quality of a decision, $R_{m,b}$ is used, $\Pi_{m,b}$ is a set policy, and $\gamma_{m,b}$ is the discount factor [34]. In each slice of the RAN, the MDP problem is solved using the F-A2C method. In ME-FDRL-RA, each slice in a RAN is considered as an F-A2C agent. Let $\Phi_b(t) = \{U^b_{M_I}(t), U^b_{M_{II}}(t), h^b_{M_{II}}(t), r^b_M(t), \Theta_b(t), P^{total}_b(t)\}$ be the shared state between the slices at time t in RAN b, which is stored in the shared account book of the RAN and updated each time step. Also, $s_{m,b}(t)$ denotes the state space at time t for slice m in RAN b, which equals $\{U_{m,b}(t), h^{Total}_{m,b}(t), r^{Total}_{m,b}(t), \Theta_b(t), P^{total}_{m,b}(t)\}$. Our proposed method is a decentralized method in which each slice makes the decision on the resource allocation to its users and the acceptance/rejection of new user requests independently of other slices in the RAN. Let $a_{m,b}(t) \in A_{m,b}$ be the selected action for slice m in RAN b at time t. It takes a value of either 0 or 1, where $a_{m,b}(t) = 1$ denotes the acceptance of a new incoming request and $a_{m,b}(t) = 0$ denotes the rejection of an incoming user's request in slice m of RAN b at time t.

The pseudocode of ME-FDRL-RA is shown in Algorithm 2. The algorithm describes the F-A2C agents' tasks. Line 1 specifies how many RANs and slices there are. In Line 2, a resource lock is defined to access the shared resources in each RAN. Also, in Line 3, the values of $\gamma_{m,b}$, $\alpha^{m,b}_a$, and $\alpha^{m,b}_c$ specify the number of time steps, the discount factor, the learning rates for actor and critic networks, respectively. We start the F-A2C algorithm for each slice in each RAN in Line 4. In Lines 5–6, the initial weight values for $\theta^{m,b}_a$ and $\theta^{m,b}_c$ are determined, respectively. Also, counter t is defined and initialized to count the time steps in Line 7. In the F-A2C algorithm, for a state $s_{m,b}(t) \in S_{m,b}$, a possible action is selected by the agent according to its policy $\pi_{m,b}$ at time-step t where $\pi_{m,b} \in \Pi_{m,b}$. The initial state $s_{m,b}(t)$ is received from the environment in Line 8. For users of rate-based slices, the IA-EPA algorithm is invoked in Line 11. Then, the selected action in Line 9 is applied to the environment with acquiring the resource lock in Lines 12–13. In Line 14, the slice releases the resource lock for other slices' usage in the RAN. As a result of the applied action, the environment changes to the next state $s_{m,b}(t+1)$, and the agent can measure the quality of the applied action with reward $R_{m,b}(t)$ (Line 15). We consider the following cumulative reward function in our method:

$$R_{m,b}(t) = |ln(r^{Total}_{m,b}(t) - h^{Total}_{m,b}(t) + 1)| + B(D_{m,b}(t)) \quad (12)$$

where $|ln(r^{Total}_{m,b}(t) - h^{Total}_{m,b}(t) + 1)|$ is the reward function for constraint (4b) and $B(D_{m,b}(t))$ is the reward function for constraint (4d) [34]. If the slice isolation condition is satisfied, then $B(D_{m,b}(t))$ is equal to 0, otherwise, $B(D_{m,b}(t))$ is equal to $w_{m,b}(w_{m,b} > 0)$, with an adjustable value for each slice during simulation. It is clear that F-A2C agents converge when the reward of all agents converges to zero. The advantage of the applied action at time t is calculated in Line 16, where $\gamma_{m,b} \in [0, 1]$ denotes the discount factor for slice m in RAN b. Then, we update the actor and critic networks in Lines 17–18. In Line 19, the next action according to the slice policy is selected for the state $s_{m,b}(t+1)$. In Lines 20–22, at the end of each PW, each agent sends weights of its actor and critic networks to the Federated-Aggregator in CU. Then, the Federated-Aggregator collects the weights and calculate the average of the weights and sends back the average to the agents (Lines 29–33). In Line 23, each agent updates its actor and critic networks with new weights. At the end of each PW, we invoke the SBiLSTM model to predict the allocated resources in the next PW, then, we update the allocated resources for slice m in RAN b with acquiring the resource lock (Lines 24–27). In Line 28, the next time step begins. Following is a general summary of the state space, action space, and cumulative reward function:

State space: $\{U_{m,b}(t), h^{Total}_{m,b}(t), r^{Total}_{m,b}(t), \Theta_b(t), P^{total}_{m,b}(t)\}$
Action space: $a_{m,b}(t) = \{0, 1\}$
Reward Function: $R_{m,b}(t) = |ln(r^{Total}_{m,b}(t) - h^{Total}_{m,b}(t) + 1)| + B(D_{m,b}(t))$

5.4. Computational complexity analysis

To obtain the computational complexity of the ME-FDRL-RA method, we need to determine the complexities of F-A2C, IA-EPA, and SBiLSTM methods. We have used neural networks to implement the actor-critic networks in ME-FDRL-RA. The resource allocation in a slice is managed by a dedicated F-A2C agent, which is executed on a CPU thread independently of other agents. Thus, we have parallel F-A2C agents, decreasing the time and computational complexity of our proposed method. The computational complexity of F-A2C agents in all RANs can be expressed as follows:

$$O\left(\frac{B \times M_b \times T_{max}(\sum^{L_a}_{i=0} un^{(i)}_a un^{(i+1)}_a + \sum^{L_c}_{i=0} un^{(i)}_c un^{(i+1)}_c)}{B \times N_{Th}} + \frac{T_{max}}{PW} \times A\right) \quad (13)$$

where B is the number of RANs, M_b is the number of F-A2C agents for slices in RAN b, N_{Th} is the number of CPU threads and T_{max} is the required number of steps during training. Also, L_a denotes the number of layers in the actor network, and L_c denotes the number of layers in the critic network. Furthermore, $un^{(i)}_a(un^{(j)}_a)$ and $un^{(i)}_c(un^{(j)}_c)$ are the number of units in the ith (jth) layer of the actor and critic networks, respectively. Considering that DRL is implemented in DU and FL in CU, and both are located inside the BBU, two cases arise to calculate the computational complexity of the aggregation operation:

Algorithm 2: Mobility Aware and Energy-Efficient Federated Deep Reinforcement Learning Assisted Resource Allocation (ME-FDRL-RA) Method for 5G-RAN Slicing

Input: The number of RANs, the number of slices in each RAN, $\gamma_{m,b}$, T_{max}, $\theta_a^{m,b}$, $\theta_c^{m,b}$, $\alpha_a^{m,b}$, and $\alpha_c^{m,b}$
Output: Optimized $\theta_a^{m,b}$ and $\theta_c^{m,b}$
Determine the number of RANs and the number of slices in each RAN
Define resource lock $L_{resource}^{m,b}$ in each RAN
Initialize T_{max}, $\gamma_{m,b}$, $\alpha_a^{m,b}$, and $\alpha_c^{m,b}$
for *Every slice in each RAN* **do**
  Initialize actor network with random parameters $\theta_a^{m,b}$
  Initialize critic network with random parameters $\theta_c^{m,b}$
  Initialize slice step counter $t = 1$
  Get initial state $s_{m,b}(t)$
  Choose action $a_{m,b}(t) = \pi_{m,b}(s_{m,b}(t))$ according to $\pi_{\theta_a^{m,b}}(a_{m,b}(t)|s_{m,b}(t))$
  while $t <= T_{max}$ **do**
    Invoke IA-EPA for rate-based slices to determine the required resources
    Acquire $L_{resource}^{m,b}$
    Implement action $a_{m,b}(t)$
    Release $L_{resource}^{m,b}$
    Measure reward $R_{m,b}(t) = V_{\theta_c^{m,b}}(s_{m,b}(t))$, next state $s_{m,b}(t+1)$
    Calculate $\delta = R_{m,b}(t) + \gamma_{m,b} V_{\theta_c^{m,b}}(s_{m,b}(t+1)) - V_{\theta_c^{m,b}}(s_{m,b}(t))$
    Update actor: $\theta_a^{m,b} = \theta_a^{m,b} + \alpha_a^{m,b} \nabla_{\theta_a^{m,b}} log \pi_{\theta_a^{m,b}}(a_{m,b}(t)|s_{m,b}(t))\delta$
    Update critic: $\theta_c^{m,b} = \theta_c^{m,b} + \alpha_c^{m,b} \delta \nabla_{\theta_c^{m,b}} V_{\theta_c^{m,b}}(a_{m,b}(t)|s_{m,b}(t))$
    Choose action $a_{m,b}(t+1) = \pi_{m,b}(s_{m,b}(t+1))$ according to $\pi_{\theta_a^{m,b}}(a_{m,b}(t+1)|s_{m,b}(t+1))$
    if $t \ \% \ PW == 0$ **then**
      Send $W_{\theta_a^{m,b}}$ and $W_{\theta_c^{m,b}}$ to Federated-Aggregator in CU
      Wait for Federated-Aggregator
      Update $\theta_a^{m,b}$ and $\theta_c^{m,b}$ with new weights $\bar{W}_{\theta_a^{m,b}}$ and $\bar{W}_{\theta_c^{m,b}}$
      Invoke SBiLSTM to determine $\bar{r}_{m,b}(t)$ in the next PW
      Acquire $L_{resource}^{m,b}$
      Update the allocated resources for the slice with $\bar{r}_{m,b}(t)$
      Release $L_{resource}^{m,b}$
    $t = t + 1$
`Federated-Aggregator(`$W_{\theta_a^{m,b}}$, $W_{\theta_c^{m,b}}$`) :`
  Collect $W_{\theta_a^{m,b}}$ and $W_{\theta_c^{m,b}}$ from each slice in each RAN
  Calculate $\bar{W}_{\theta_a^{m,b}}$: the average of collected $W_{\theta_a^{m,b}}$
  Calculate $\bar{W}_{\theta_c^{m,b}}$: the average of collected $W_{\theta_c^{m,b}}$
  return $\bar{W}_{\theta_a^{m,b}}$, $\bar{W}_{\theta_c^{m,b}}$

- Case 1 - BBU centralized design: In this case, DU and CU act as two functions of BBU, and CU only averages the weights of similar agents from RANs and returns the result to each agent. Since each agent is processed in parallel in our design, its computational complexity will be $A = O(BLQ)$, where B is the number of RANs, L is the number of agent layers, and Q is the averaging cost.
- Case 2 - BBU distributed design: In this case, CU and DU are physically separated from each other, and the cost of signaling to send the weights of each slice and return the result to the slices must be added to the operating cost of the first case.

It is noted that we have considered case 1. Ideally, we would have a sufficient number of CPU threads N_{Th} to allow each F-A2C agent to execute in parallel, resulting in a time complexity as low as possible, so, the computational complexity of all F-A2C agents is $O(T_{max}(\sum_{i=0}^{L_a} un_a^{(i)} un_a^{(i+1)} + \sum_{i=0}^{L_c} un_c^{(i)} un_c^{(i+1)}))$.

The authors in [34] show that the computational complexity of a SBiLSTM model equals $W = O((4n_c n_c + 4n_i n_c + n_c n_o + 3n_c)N_{ep})$, where n_c is the number of memory cells, n_i is the number of input units, n_o is the number of output units, and N_{ep} denotes the number of epochs. Therefore, the computational complexity of all SBiLSTM models can be calculated as follows:

$$O(\frac{B \times M_b \times \frac{T_{max}}{PW} \times W}{B \times N_{Th}}) \tag{14}$$

As a result, the computational complexity of all SBiLSTM models is $O(\frac{T_{max}}{PW} \times W)$.

The IA-EPA method in Algorithm 1 consists of three loops. The first loop is for finding the number of idle RBs (N). Then, for the selected RBs in the first loop, two loops are executed to find the optimal answers every time. So, the computational complexity of the IA-EPA method equals $O(N I_{max} L_{max})$, where I_{max} is the maximum number of iterations for the outer loop and L_{max} is the maximum number of iterations for the inner loop. Considering the SBiLSTM method is executed per PW and the IA-EPA method is executed for users of the rate-based slices, we can obtain the total computational complexity of the ME-FDRL-RA method in the worst case with multiple CPU threads as follows:

$$O(T_{max}(\sum_{i=0}^{L_a} un_a^{(i)} un_a^{(i+1)} + \sum_{i=0}^{L_c} un_c^{(i)} un_c^{(i+1)} + W + A + N I_{max} L_{max})) \tag{15}$$

Compared to the sequential methods in [9–19], the ME-FDRL-RA method uses parallel and distributed execution to handle the slices. Also, each slice in the ME-FDRL-RA method is managed by an F-A2C agent, which runs on a CPU thread independently of other agents. So, it can simplify managing (creating, deleting, starting, stopping) the slices than [20–25] and [27–30]. According to (15), growing the number of slices will not affect the computational complexity of the algorithm, since it will not increase the size of actor-critic networks. However, the time and computational complexity of ME-FDRL-RA is similar to [34] and less than [35,36] that use a federated manner in each time step to speed up convergence. Therefore, the ME-FDRL-RA method is scalable in terms of time and computational complexity.

5.5. Scalability assessment and signaling overhead

Dynamic slice creation and management is a main objective of RAN slicing that must be considered in each proposed RAN slicing. To achieve this objective,in our proposed method, each RAN is managed by a controller in the DU (see Fig. 2). Each DU-Controller is responsible for executing the resource allocation algorithm, including F-A2C agents, IA-EPA methods, and SBiLSTM models in each RAN. Each slice has its own F-A2C agent, IA-EPA method, and SBiLSTM model. We consider a dedicated CPU thread for managing a slice of a RAN in DU-Controller, so that each thread computes its own computations separately of other threads of the DU-Controller. Therefore, we can create and manage the slices dynamically, while other methods respond to service requests for the slices sequentially [9–19]. Compared to the ME-FDRL-RA, other methods such as [20–25] and [27–30] need to reconfigure the learning network and start the learning process from the beginning. The ME-FDRL-RA method has a few distinctive advantages, despite its scalability being similar to [26,31,33,34], and [37].

To assess the signaling overhead in the ME-FDRL-RA method, we consider three general deployment scenarios in accordance with our distributed RAN architecture. The scenario of case 1 is used to implement our method in this paper.

Case (1) single system/multiple threads: In this case, we have sufficient computational resources to assign to the slices of each DU-controller. This means that all F-A2C agents, SBiLSTM models, and IA-EPA methods in DU and federated aggregator in CU are executed on one system, using different CPU threads. Therefore, we do not have any signaling messages among slices by front-haul and back-haul links. Indeed, we have some internal messages between F-A2C agents in a RAN, which can be exchanged via the shared account book. We use a semaphore (CPU lock) to access this shared account book and shared resources to minimize the signaling overhead between the different

slices in a RAN. Also, given that the F-A2C agents in DU controllers and the federated aggregator in CU are placed in one system, they do not use the back-haul link to exchange the messages.

Case (2) multiple systems/multiple threads: In this case, DU controllers and federated aggregator are placed on different systems and each slice in a controller is mapped to a CPU thread. The signaling overhead for the slices into a DU controller is similar to that in case 1. Our method does not impose any signaling messaging overhead for communication between the DU controllers. Also, each slice in each controller sends its networks' weights to the federated aggregator in CU and receives the new weights via mid-haul links.

Case (3) single system/single thread: In this case, DU controllers and federated aggregator are placed on one system and a single thread is used to implement the resource allocation algorithms, similar to methods in [9–19]. Each entity use the computational resources sequentially. Although the signaling overhead for the slices inside a DU controller is similar to case 1, the scalability of this case is very low.

The ME-FDRL-RA is able to support the above scenarios. However, the signaling in case 1 is similar to [33,34]. Also, the signaling in case 2 is much lower than other distributed methods such as [35,36], which use the federated aggregator at each time step.

6. Simulation results

In this section, we first explain the simulation setup and introduce the competitive methods and evaluation parameters. Then, we analyze the simulation results.

6.1. Simulation setup

ME-FDRL-RA is an MDP-based solution for resource allocation problems in Section 4 and its performance improves over time with receiving proper feedback from the system. Our simulation includes a multi-RAN scenario with 4 RANs. Poisson distribution is used to model users' requests in slice m of RAN b with arrival and departure rates $\lambda[m,b]$ and $\mu[m,b]$. The generated requests are assigned to the users according to a FIFO (First-In-First-Out) policy. When each user releases its current resources, it remains in the system until the next request is assigned to that user. This operation repeats forever until the end of the simulation. As a result, we are faced with time-varying requests from users to the system. It should be noted that we needed to have a fixed total number of users due to the requirements of the mobility model (i.e., SWIM) and simulate a time-varying request rate by re-assigning requests to the same user according to a FIFO strategy. In F-A2C, we have considered the request rejection penalty to be twice the isolation violation penalty in order to serve more users. Also, we consider 10 slices in each RAN, in which 4 slices are resource-based slices and 6 slices are rate-based slices. We use the same simulation parameters for considered competitive methods. It is shown in [41,42] that using federated averaging can increase the convergence speed and improve decision-making in F-A2C methods. The convergence of methods that use federated averaging for models with non-identical data distribution is an important challenge, despite the proposed solutions to tackle this problem. Here we use an identical data distribution in F-A2Cs and the use of the simple form of federated averaging can help to increase the convergence speed of ME-FDRL-RA. Also, the authors in [43] demonstrate that the average of weights leads to better performance in identical models. In our particular implementation, we have 4 RANs and each RAN accommodates 10 slices. For each aggregation operation, FDRL uses 4 agents. In the case of slice 1, it aggregates (using FedAvg) the model of slice 1's agent in RAN #1, slice 1's agent in RAN #2, slice 1's agent in RAN #3, and slice 1's agent in RAN #4). To guarantee the convergence condition of actor and critic networks in F-A2Cs, we consider the learning rates of actor and critic networks to be very small [33], and [42]. Our simulation includes 48 000 time steps and a prediction window of 1200 time steps. Moreover, the speed of human movement in the SWIM mobility model equals 1.4 m/s [44]. Each RAN has 200 users, whose home locations are chosen randomly according to a uniform distribution in the RAN. In addition, when a mobile user reaches her/his desired destination, she/he will choose her/his pause time randomly based on a uniform distribution from the designated interval. Table 2 lists the simulation's used parameter values for the system and mobility model. Based on a comprehensive survey of the state-of-the-art work in the literature, we have chosen the parameters that are feasible in practice and consistent with the literature. In particular, we have adopted the system model parameters from Refs. [33,34], and the mobility-relevant parameters from [40].

We used Python 3.7 including Gym, Keras, and TensorFlow libraries for creating the environment and the agents. Also, we ran the simulations in PyCharm IDE.

6.2. Competitive methods

We compare ME-FDRL-RA with five other methods. The competitive methods are as follows:

- **Extended EE-DRL-RA:** The EE-DRL-RA [34] method allocates the radio and power resources to resource-based and rate-based slices in a RAN in which A3C is used to decide on allocating resources on a small time-scale and SBiLSTM is used to allocate resources to the slices on a large time-scale. In this method, each slice is implemented on a separate CPU thread and multiple A3C blocks in [34] can run concurrently to speed up convergence and enhance decision-making in A3C and SBiLSTM methods. It uses a method called EE-PA for dynamic power allocation taking into account energy efficiency and channel condition. However, the EE-DRL-RA method is not designed for multi-RAN scenarios and does not consider the mobility of the users and inter-RAN frequency interference in the RAN resource allocation. Hence, we used a promoted version of EE-DRL-RA called Extended EE-DRL-RA to adjust to our current multi-RAN scenario. Extended EE-DRL-RA applies the mobility-aware feature in SBiLSTM and the IA-EPA method for resource allocation to users of rate-based slices in the multi-RAN scenario.
- **Extended iRSS:** Compared to [34], the iRSS method [33] allocates a fixed amount of power to users of the slices and uses the conventional LSTM model for allocating resources on a large time-scale. We implemented a multi-RAN version of this method called Extended iRSS that takes into account inter-RAN interference in our simulation.
- **E-FDRL-RA:** To evaluate the impact of not considering the mobility of users in the proposed method, we have not considered the traffic belonging to mobile users in determining the allocated resources to slices in SBiLSTM. We will refer to this variation as E-FDRL-RA.
- **ME-DRL-RA:** To evaluate the impact of not considering the federated averaging in the proposed method, we implemented ME-DRL-RA in which A2C is used rather than F-A2C in each agent.
- **Static method:** To evaluate the proposed method, we have considered a benchmark method called the Static method, which assigns a fixed amount of resources to each network slice, regardless of the dynamic changes in the network environment and user demands. According to the Static method, we have divided the resources equally between each slice.

6.3. Evaluation parameters

In order to assess the performance of the proposed ME-FDRL-RA method, we compare the ME-FDRL-RA against methods in Section 6.2 in terms of (i) the prediction accuracy of SBiLSTM (ii) the convergence speed (iii) the energy efficiency (iv) the user acceptance percentage (v) the utilization (vi) the isolation. (vii) the mobility awareness The definitions of the evaluation metrics are summarized as follows:

Table 2
Simulation parameters

System model parameters	Value
Number of RANs	4
Number of slices in each RAN	10
Number of resource-based slices in each RAN	4
Number of rate-based slices in each RAN	6
Learning rate of actor networks $\alpha_a^{m,b}$	10^{-4}
Learning rate of critic networks $\alpha_c^{m,b}$	10^{-4}
Discount factor	0.99
Number of RBs in each RAN (Θ_b)	200
Total Power of RRUs in each RAN ($P_{total,b}$)	2000 W
P_{max}	40 W
Prediction Window (PW)	1200 s
Confidence level for the SBiLSTM model	97%
Required RBs for resource-based slices in each RAN	$\{4, 3, 2, 1\}$ RBs
Required rate for rate-based slices in each RAN	$\{8, 7, 6, 4, 3, 2\}$ Mbps
Minimum threshold of the length of duration to ensure isolation ($T_{m,b}^{th}$)	PW/10
$\lambda[m, b]$	[0.7, 0.75, 0.6, 0.75, 0.65, 0.75, 0.6, 0.8, 0.7, 0.75]
$\mu[m, b]$	[0.75, 0.85, 0.75, 0.85, 0.75, 0.85, 0.76, 0.85, 0.75, 0.85]
RB bandwidth	1.8 kHz
Path loss exponent (n)	2
System losses (C)	38.4
SWIM mobility model parameters	**Value**
Total users	800
The number of fixed users	720
The number of mobile users	80
k	0.03
α	0.6
Speed	1.4 m/s
Cell size	300 m
RAN size	1200 × 1200 m^2
Pause time	Randomly selected value between 5 and 20 s

- **SBiLSTM accuracy:** Our main metric for assessing SBiLSTM's accuracy in predicting $r_{m,b}^{Total}(t)$ is the error between estimated and actual values.
- **Convergence speed:** To measure the convergence speed, we consider the convergence of three methods used on the small and large time-scales and the resource determination method for rate-based slices. In general, the convergence of these three methods leads to the convergence of the overall method. Indeed, the method converges when there is no improvement in the model and the reward decreases to zero.
- **Energy efficiency and user acceptance percentage:** A high convergence speed leads to proper decision-making on resource allocation, which in turn increases the user acceptance percentage and the total energy efficiency per PW. We calculate the energy efficiency (EE) of each accepted user by Eq. (7a), and the percentage of accepted users by the ratio of accepted users to total incoming users per PW.
- **Utilization:** We calculate the utilization of the RBs in each slice of each RAN during a PW by $\sum_t h_{m,b}^{Total}(t) / \sum_t r_{m,b}^{Total}(t)$.
- **Isolation:** The isolation degree is our metric to measure the isolation among slices and equals the number of time steps of a PW that $r_{m,b}^{Total}(t)$ of slice m in RAN b has not changed.
- **Mobility Awareness:** We evaluate the effect of mobility of the users in the proposed method with different mobility degrees.

6.4. Simulation analysis

In this section, we analyze the simulation results based on the aforementioned evaluation parameters as follows:

6.4.1. Prediction accuracy of SBiLSTM

First, the SBiLSTM model is examined on a large time-scale prediction accuracy basis due to its effect on isolation among slices. Therefore, we evaluate the performance of the SBiLSTM without FDRL and IA-EPA methods. We demonstrate the errors between the targets and the outputs of training data, validation data, and test data are shown in Fig. 3. In this simulation, 70%, 15%, and 15% of the dataset are utilized for training, validation, and testing, respectively. The figure shows that the frequency of most errors between targets and outputs is around zero, which shows its prediction accuracy. It should be mentioned that the authors in [34] show that the SBiLSTM model is superior to conventional LSTM models for traffic prediction.

6.4.2. Convergence speed

This section aims to examine how methods that are used on a small time-scale resource allocation (i.e., A3C, A2C, and F-A2C) and on a large time-scale resource allocation (i.e., conventional LSTM and SBiLSTM) affect the convergence speed of the overall method in the multi-RAN scenario. Also, we show the proposed IA-EPA method that converges in a few iterations.

Fig. 4 displays the average cumulative rewards for all RANs in the five comparable methods per PW. From the figure, we can see that each of the five methods eventually converges to zero. We may face a high reward at the start of the simulation because of an inexperienced or failed exploration, or a fluctuation in the LSTM models because of the lack of sufficient input data. However, ME-FDRL-RA shows a better performance than the compared methods over time. It is noted that F-A2Cs in ME-FDRL-RA and E-FDRL-RA use the obtained knowledge from all RANs and A3C in extended EE-DRL-RA and extended iRSS use the obtained knowledge from A3C blocks in a RAN. So, the federated averaging methods have fast state space exploration and convergence speed compared to the other methods. Also, although increasing the number of the A3C blocks leads to a fast exploration of the space states and improves the decision-making on resource allocation, they will require more CPU threads to be assigned to the slices. In other words, extended EE-DRL-RA and extended iRSS use two A3C blocks in each RAN, so, they explore system space with twice the number of CPU threads used in other methods. The ME-FDRL-RA method has lower cumulative rewards than other methods because of considering the traffic of the mobile users in its SBiLSTM and using the knowledge obtained by other RANs through federated averaging. Also, it has lower slice isolation violations and learns the optimal allocation policy faster

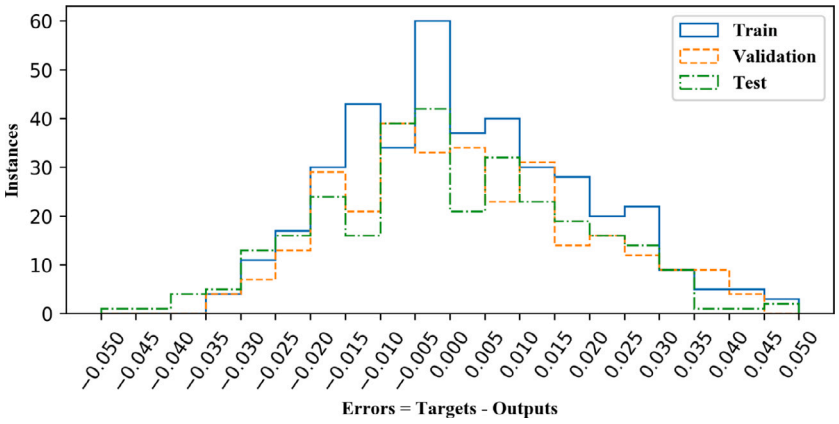


Fig. 3. Traffic prediction accuracy for SBiLSTM model on a large time-scale.

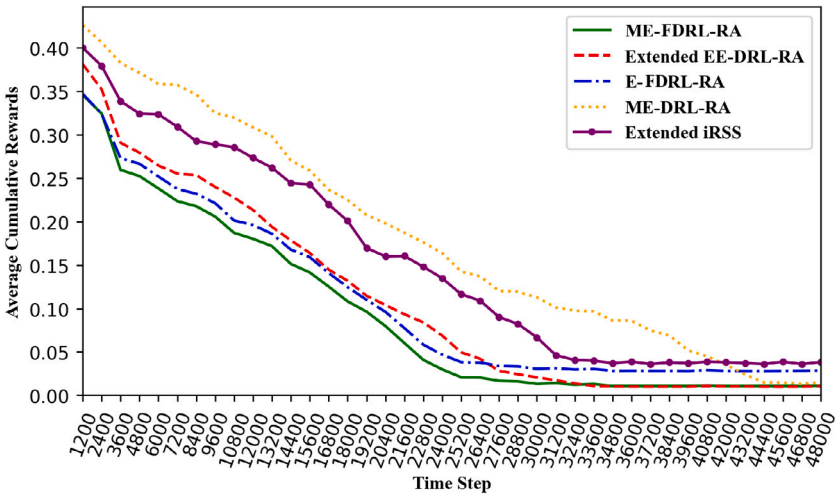


Fig. 4. Convergence speed for the compared methods in the multi-RAN scenario.

than the competitive methods. The E-FDRL-RA does not consider the traffic of mobile users in its SBiLSTM, so, it violates its own isolation and needs to ask for more resources from the residual resources and has higher cumulative rewards than ME-FDRL-RA. The extended EE-DRL-RA uses two A3C blocks in a RAN to explore the state space and a shared SBiLSTM between the blocks. Therefore, it has higher penalties than ME-FDRL-RA and ME-FDRL-RA at the beginning of the simulation because of unsuccessful exploration. However, it has a lower penalty than E-FDRL-RA at the end of the simulation because of the prediction accuracy in its shared SBiLSTM and considering the mobile users' traffic. Also, extended iRSS uses the conventional LSTM for traffic prediction on a large time-scale and two A3C blocks in a RAN to explore the state space, but it does not consider the mobile users' traffic. Therefore, it has higher penalties for isolation violations and incorrect decision-making on resource allocation to users. The ME-DRL-RA method does not use federated averaging, which leads to the wrong rejection of users' requests and decreases the convergence speed. Overall, ME-FDRL-RA explores the state space very fast and learns the optimal policy faster than the compared methods. Therefore, we can use ME-FDRL-RA as a feasible online solution for multi RAN slicing.

Moreover, the convergence speed of the IA-EPA solution for determining the required power and RBs for the users in the rate-based slices taking into account channel conditions and frequency interference among the users in different RANs is shown in Fig. 5. In the figure, P^* and EE^* are the optimal solutions for the optimization problem in Eqs. (7a), (7b), and (7c), which are calculated by successive linear programming (SLP) in Lingo (optimization problem solver). I represents the total number of iterations (the outer loop counter) in the IA-EPA algorithm. The figure illustrates that IA-EPA converges to the optimal solution within a few iterations. As a result, the IA-EPA method is a scalable method that converges to the optimal solution with low costs.

6.4.3. Energy efficiency and user acceptance percentage

In Figs. 6(a) and 6(b), we investigate the energy efficiency (EE) and the accepted users' percentage in the six competitive methods over time. We use a constant amount of power and RBs for the users in the resource-based slices. Also, the required resources for the users in the rate-based slices are determined by the IA-EPA method by considering the channel condition and frequency interference between the users in different RANs. Using the maximum power for the resource-based slices increases the power usage and the interference on the same RBs in neighbor's RANs. Increasing the interference leads to an increase in power and RBs usage for satisfying the optimization problem (7) in the rate-based slices. Consequently, more users may be rejected because of lack of resources. As shown in the figures, ME-FDRL-RA and E-FDRL-RA have a higher user acceptance rate than other methods over time because they are able to make resource allocation decisions using the obtained knowledge from other RANs through federated averaging. As a result, increasing the percentage of user acceptance leads to an increase in the amount of the total EE of users per PW. Although extended EE-DRL-RA and ME-DRL-RA have a low user acceptance rate and lower EE at the beginning of the simulation, they accept more users with learning the optimal allocation policy over time, which leads to

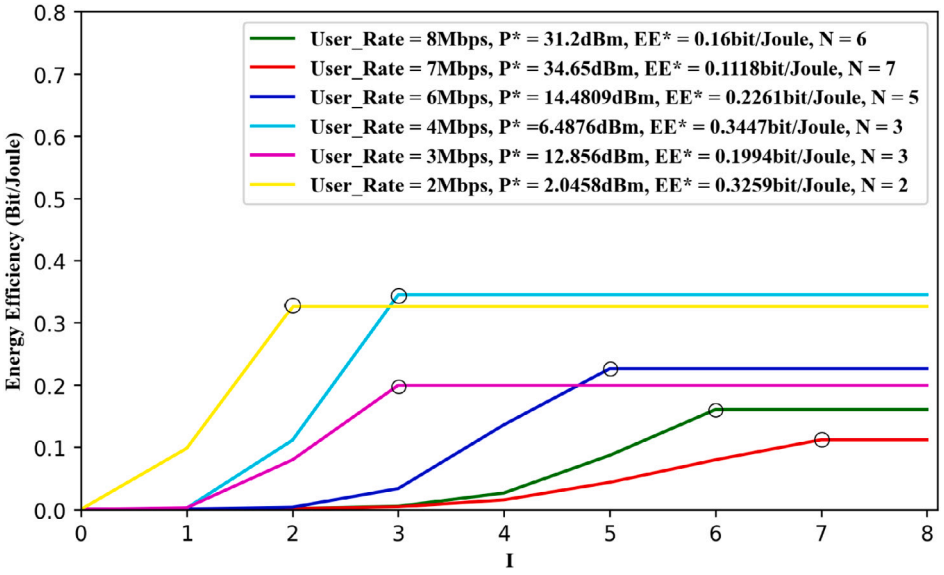
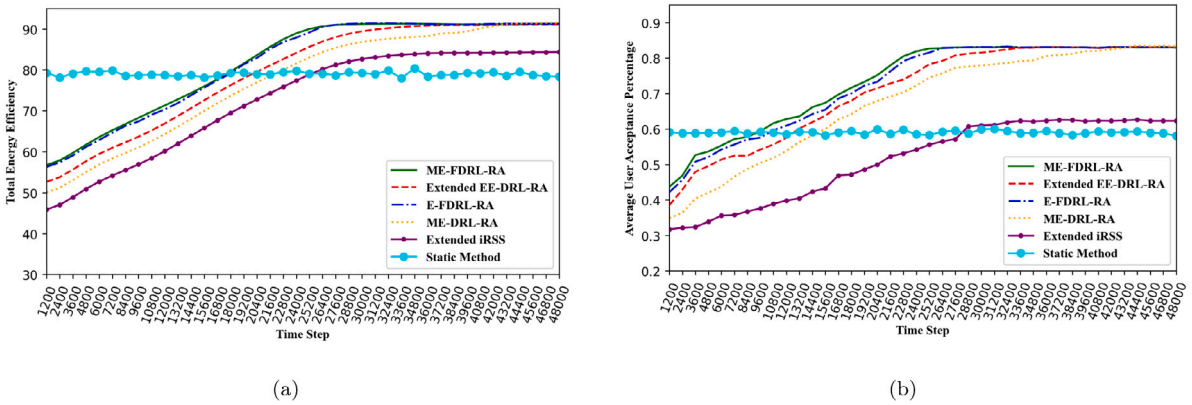


Fig. 5. Energy efficiency versus number of iterations with different transmission rates in IA-EPA method.



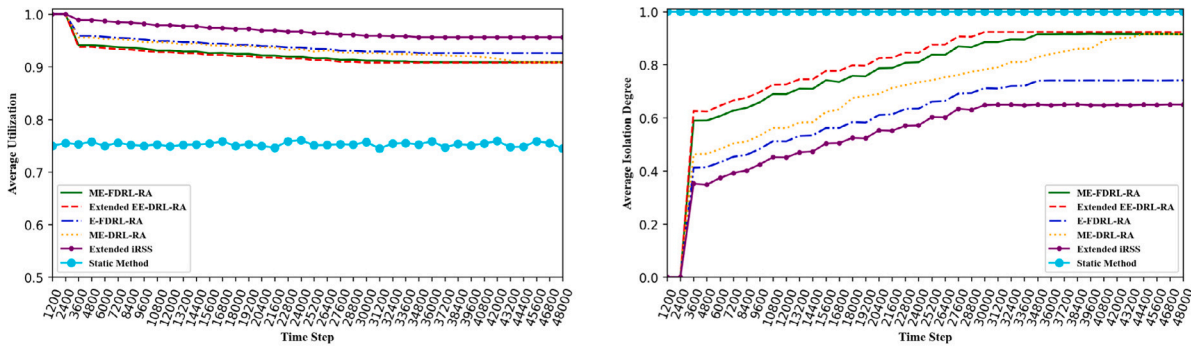
(a) (b)

Fig. 6. (a) Total energy efficiency (b) Average user acceptance percentage.

an increase in EE. Also, extended iRSS and static method uses a fixed amount of power for all users in the slices, which will lead to resource shortages and the rejection of more users. Therefore, extended iRSS takes lower energy efficiency and user acceptance rate than other RL methods. The static method cannot adapt to the dynamic changes in the network environment and user demands, such as mobility, channel variations, interference, and traffic fluctuations. For example, suppose a slice experiences a sudden increase or decrease in its traffic load due to mobility or channel variations. In that case, it may be unable to adjust its resource allocation accordingly. This condition may lead to performance degradation or resource wastage. Therefore, this method has a lower user acceptance rate than other methods. Since the total EE is directly related to the user acceptance rate, this method also has a lower EE than the other methods.

6.4.4. Utilization vs. Isolation

Next, we examine the correlation between resource utilization versus isolation degree in Figs. 7(a) and 7(b). Both the resource utilization and the isolation degree depend on the prediction accuracy of the LSTM models. In the simulation, we do not use the LSTM in the first two PWs because sufficient data does not exist for estimating $r_{m,b}^{Total}(t)$. Thus, to predict $r_{m,b}^{Total}(t)$ for the next PW, we collect $h_{m,b}^{Total}(t)$ as input data for each slice in the first two PWs. So, the amount of isolation degree equals zero in the first two PWs in Fig. 7(b). Also, the amount of resource utilization in the first two PWs equals 1 because the allocated resources ($r_{m,b}^{Total}(t)$) for each slice match the required resources ($h_{m,b}^{Total}(t)$) by the slice. As shown in the figure, the SBiLSTM models achieve a good performance over time by collecting more input data. It can be seen that extended EE-DRL-RA outperforms the other RL methods in terms of prediction accuracy and convergence speed due to using the shared SBiLSTM models between the A3C blocks in each RAN. Although we see that the SBiLSTM models in ME-FDRL-RA and ME-DRL-RA methods have a lower isolation degree than extended EE-DRL-RA, they utilize a low number of CPU threads. The E-FDRL-RA and extended iRSS methods have a lower isolation degree than the other RL methods because they do not consider the mobile users' traffic for resource allocation to the slices on a large time-scale and encounter the violation of isolation frequently. Also, extended iRSS uses the conventional LSTM, which has a lower prediction accuracy than SBiLSTM. Fig. 7(a) shows that the resource utilization of extended iRSS is higher than the compared RL methods. Because the isolation is violated more often in the extended iRSS method, it is possible that the slices use the residual resources frequently to serve their users. Therefore, $r_{m,b}^{Total}(t)$ will be equal to $h_{m,b}^{Total}(t)$ and it leads to higher resource utilization compared to other methods. As a result, we observe from Fig. 7(b) that SBiLSTM can increase the isolation among the slices while maintaining the resource utilization at an acceptable level. Furthermore, due to the static method having a fixed amount of RBs over the simulation time, its isolation equals 1 (Fig. 7(b)). However, it may cause resource under-utilization or over-utilization, depending on the actual traffic load of each slice.



(a) (b)

Fig. 7. (a) Utilization of slices (b) Isolation degree among slices.

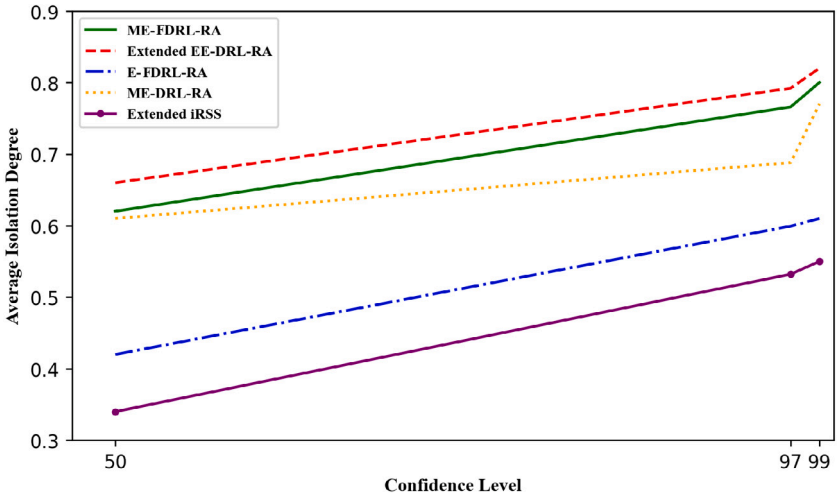


Fig. 8. The effect of confidence level on isolation degree in a large time-scale.

For example, if a slice has a low traffic load but a high resource allocation, it may waste the resources that could be used by other slices. On the other hand, if a slice has a high traffic load but a low resource allocation, it may suffer from poor performance and quality of service. In Fig. 7(a), the utilization of this method is lower than other methods, which shows the resources wasted in the slices.

We have defined a confidence level in the large time-scale resource prediction to determine the upper bound of the $r_{m,b}(t)$. Fig. 8 shows the relationship between the isolation of the slices versus the confidence level. As shown in the figure, increasing or decreasing the confidence level impacts on the isolation of the slices. Increasing the confidence level leads to increasing the amount of allocated resources for a slice and enhances the isolation of the slice. On the other hand, it may decrease the resource utilization. Slice isolation can also be violated frequently when the confidence level decreases. The figure shows that extended EE-DRL-RA outperforms compared to the competitive methods in terms of the isolation degree due to using the shared SBiLSTM methods in each A3C block in each RAN. Fig. 8 demonstrates that considering the mobile users' traffic in extended EE-DRL-RA, ME-FDRL-RA, and ME-DRL-RA leads to a higher isolation degree than E-FDRL-RA and extended iRSS. Also, extended iRSS has a lower isolation degree than other methods because it does not reserve resources for the mobile users and uses the conventional LSTM. Thus, extended iRSS needs more resources from the residual resources to serve its users and cannot maintain the slices' isolation for longer periods of time.

6.4.5. Mobility awareness

We examine the effect of the mobility degree for human mobile users in SWIM on the proposed method in Fig. 9. In our mobility model, for a small amount of α, the user inclines to go to popular places and for a large amount of α, the user will be more inclined to visit places near its home. As mentioned earlier, extended iRSS, extended EE-DRL-RA, and static method do not consider user mobility among RANs, and in ME-DRL-RA, the decisions are subject to fluctuations. Therefore, we compare ME-FDRL-RA and E-FDRL-RA in terms of isolation degree and utilization with different alphas in the figure to show that the proposed method can handle different degrees of mobility. As you see, the proposed ME-FDRL-RA method outperforms E-FDRL-RA in different conditions of the mobility of users in terms of isolation degree. Also, the utilization of ME-FDRL-RA is nearly the same as E-FDRL-RA, which shows that the allocated resources in the SBiLSTM method for ME-FDRL-RA are close to the required resources. In addition, we show the impact of the number of mobile users on the proposed method in Fig. 10. As mobile users typically increase inter-RANs mobility, slice isolation and slice utilization may be affected. However, Fig. 10 demonstrates that ME-FDRL-RA can assign resources to slices appropriately as the percentage of mobile users increase, and it can achieve a better balance between slice isolation and slice utilization than E-FDRL-RA.

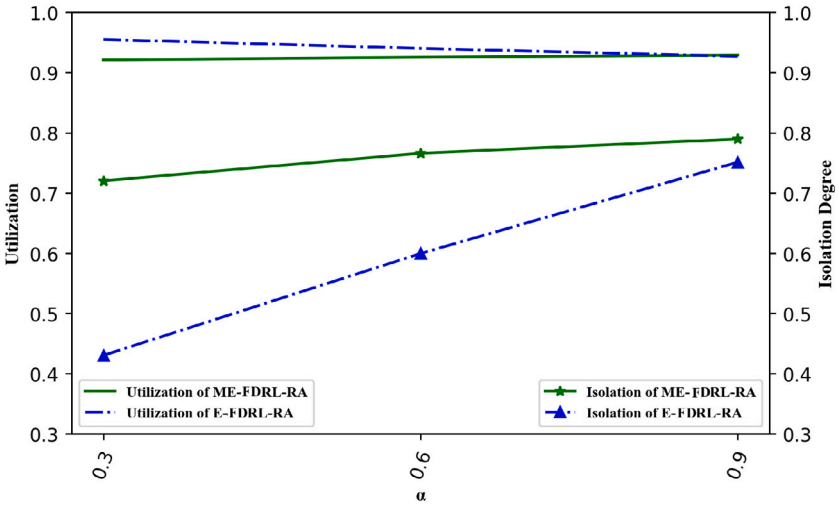


Fig. 9. The effect of SWIM mobility degree in ME-FDRL-RA.

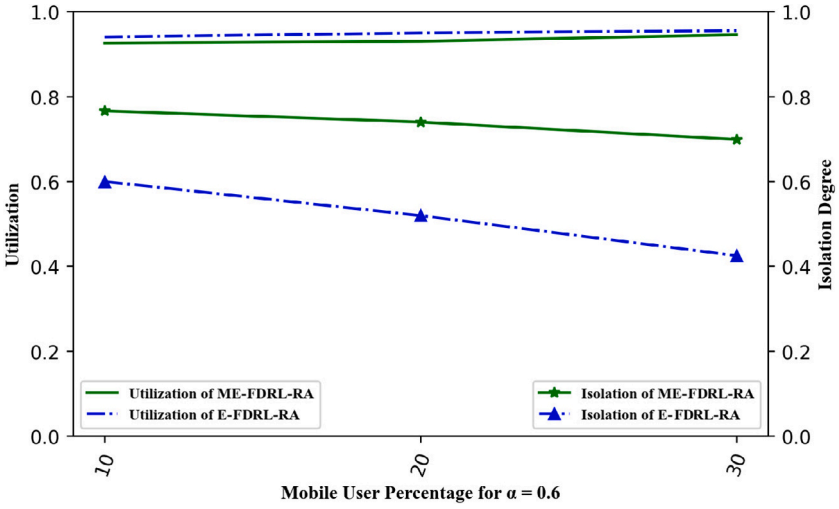


Fig. 10. The effect of the number of human mobile users in ME-FDRL-RA.

7. Conclusion and future work

In this paper, we provided a multi-RAN resource slicing method called ME-FDRL-RA, which is a combination of DL and FDRL. We have used F-A2C and SBiLSTM for decision-making on the small and large time-scales, respectively. To increase the convergence speed and improve the decision-making on resource allocation, we use a federated averaging mechanism using the obtained knowledge from the other RANs. This leads to increasing the user acceptance rate and the total energy efficiency in the system by taking accurate decisions on resource allocation. Also, we have provided an interference avoidance and energy-efficient method called IA-EPA to determine the required power and RBs for the users in the rate-based slices. In addition, we consider the user mobility in determining the allocated resources to the slices to improve isolation among slices. Generally, ME-FDRL-RA performs better than other competitive methods regarding convergence speed, energy efficiency, and user acceptance. Moreover, ME-FDRL-RA uses fewer CPU resources than Extended EE-DRL-RA and Extended iRSS while having an acceptable performance in terms of isolation degree.

We will consider the following avenues for future works: (1) Taking into account the non-orthogonal multiple access (NOMA) to serve more users in the slices (2) Studying the effect of non-Poisson distributions for users' rates on the resource allocation method.

CRediT authorship contribution statement

Yaser Azimi: Conducted all simulations, Contributed to the development of the research idea, Drafted the manuscript, Handled the revisions. **Saleh Yousefi:** Defined the research question, Contributed to the development of the research idea, Participated significantly in all stages of the research process, Including preparation of the main and revised versions. **Hashem Kalbkhani:** Contributed to the development of the research idea and the technical details of the physical layer, Supervised some parts of the modeling process, Simulations dealing with the physical layer. **Thomas Kunz:** Advised on the topic selection, Contributed to the development of the research idea, Guided the research process, Evaluation of proposed methods, Significantly assisted in the preparation of the paper.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] Y.-S. Chen, C.-S. Hsu, H.-C. Hung, Optimizing communication and computational resource allocations in network slicing using twin-GAN-Based DRL for 5G hybrid C-RAN, Comput. Commun. 200 (2023) 66–85.

[2] A.I. Salameh, M. El Tarhuni, From 5G to 6G—Challenges, technologies, and applications, Future Internet 14 (4) (2022) 117.

[3] K. Boutiba, A. Ksentini, B. Brik, Y. Challal, A. Balla, Nrflex: Enforcing network slicing in 5g new radio, Comput. Commun. 181 (2022) 284–292.

[4] Y. Azimi, S. Yousefi, H. Kalbkhani, T. Kunz, Applications of machine learning in resource management for RAN-slicing in 5G and beyond networks: A survey, IEEE Access 10 (2022) 106581–106612.

[5] Y. Xie, Y. Kong, L. Huang, S. Wang, S. Xu, X. Wang, J. Ren, Resource allocation for network slicing in dynamic multi-tenant networks: A deep reinforcement learning approach, Comput. Commun. 195 (2022) 476–487.

[6] Z. Kotulski, T.W. Nowak, M. Sepczuk, M.A. Tunia, 5G networks: Types of isolation and their parameters in RAN and CN slices, Comput. Netw. 171 (2020) 107135.

[7] S.E. Elayoubi, S.B. Jemaa, Z. Altman, A. Galindo-Serrano, 5G RAN slicing for verticals: Enablers and challenges, IEEE Commun. Mag. 57 (1) (2019) 28–34.

[8] M.A. Habibi, M. Nasimi, B. Han, H.D. Schotten, A comprehensive survey of RAN architectures toward 5G mobile communication system, IEEE Access 7 (2019) 70371–70421.

[9] J. Mei, X. Wang, K. Zheng, Semi-decentralized network slicing for reliable V2V service provisioning: A model-free deep reinforcement learning approach, IEEE Trans. Intell. Transp. Syst. (2021).

[10] D. Shome, A. Kudeshia, Deep Q-learning for 5G network slicing with diverse resource stipulations and dynamic data traffic, in: 2021 International Conference on Artificial Intelligence in Information and Communication, ICAIIC, IEEE, 2021, pp. 134–139.

[11] K. Suh, S. Kim, Y. Ahn, S. Kim, H. Ju, B. Shim, Deep reinforcement learning-based network slicing for beyond 5G, IEEE Access (2022).

[12] Y. Sun, Y. Wang, H. Yu, B. Guo, X. Zhang, A learning-based bandwidth resource allocation method in sliced 5G C-RAN, in: 2019 IEEE Globecom Workshops, GC Wkshps, IEEE, 2019, pp. 1–5.

[13] Y. Shi, M. Costa, T. Erpek, Y.E. Sagduyu, Deep reinforcement learning for NextG radio access network slicing with spectrum coexistence, IEEE Netw. Lett. (2023).

[14] B. Khodapanah, A. Awada, I. Viering, A.N. Barreto, M. Simsek, G. Fettweis, Slice management in radio access network via deep reinforcement learning, in: 2020 IEEE 91st Vehicular Technology Conference, VTC2020-Spring, IEEE, 2020, pp. 1–6.

[15] A. Nassar, Y. Yilmaz, Deep reinforcement learning for adaptive network slicing in 5G for intelligent vehicular systems and smart cities, IEEE Internet Things J. (2021).

[16] Y. Xu, Z. Zhao, P. Cheng, Z. Chen, M. Ding, B. Vucetic, Y. Li, Constrained reinforcement learning for resource allocation in network slicing, IEEE Commun. Lett. 25 (5) (2021) 1554–1558.

[17] M. Alsenwi, N.H. Tran, M. Bennis, S.R. Pandey, A.K. Bairagi, C.S. Hong, Intelligent resource slicing for eMBB and URLLC coexistence in 5G and beyond: A deep reinforcement learning based approach, IEEE Trans. Wireless Commun. 20 (7) (2021) 4585–4600.

[18] F. Saggese, L. Pasqualini, M. Moretti, A. Abrardo, Deep Reinforcement Learning for URLLC data management on top of scheduled eMBB traffic, 2021, arXiv preprint arXiv:2103.01801.

[19] M. Liu, Y. Wang, S. Meng, X. Zhao, S. Geng, Deep reinforcement learning-based resource allocation for smart grid in RAN network slice, in: Advances in Wireless Communications and Applications, Springer, 2021, pp. 199–215.

[20] Y. Shao, R. Li, Z. Zhao, H. Zhang, Graph attention network-based DRL for network slicing management in dense cellular networks, in: 2021 IEEE Wireless Communications and Networking Conference, WCNC, IEEE, 2021, pp. 1–6.

[21] Y. Shao, R. Li, B. Hu, Y. Wu, Z. Zhao, H. Zhang, Graph attention network-based multi-agent reinforcement learning for slicing resource management in dense cellular network, IEEE Trans. Veh. Technol. 70 (10) (2021) 10792–10803.

[22] I. Vilà, J. Pérez-Romero, O. Sallent, A. Umbert, A multi-agent reinforcement learning approach for capacity sharing in multi-tenant scenarios, IEEE Trans. Veh. Technol. 70 (9) (2021) 9450–9465.

[23] H. Zhou, M. Elsayed, M. Erol-Kantarci, RAN resource slicing in 5G using multi-agent correlated Q-learning, in: 2021 IEEE 32nd Annual International Symposium on Personal, Indoor and Mobile Radio Communications, PIMRC, IEEE, 2021, pp. 1179–1184.

[24] M. Zambianco, G. Verticale, Intelligent multi-branch allocation of spectrum slices for inter-numerology interference minimization, Comput. Netw. 196 (2021) 108254.

[25] M. Zambianco, G. Verticale, Mixed-numerology interference-aware spectrum allocation for eMBB and URLLC network slices, in: 2021 19th Mediterranean Communication and Computer Networking Conference, MedComNet, IEEE, 2021, pp. 1–8.

[26] Q. Liu, T. Han, E. Moges, EdgeSlice: Slicing wireless edge computing network with decentralized deep reinforcement learning, in: 2020 IEEE 40th International Conference on Distributed Computing Systems, ICDCS, IEEE, 2020, pp. 234–244.

[27] A.M. Nagib, H. Abou-Zeid, H.S. Hassanein, Transfer learning-based accelerated deep reinforcement learning for 5G RAN slicing, in: 2021 IEEE 46th Conference on Local Computer Networks, LCN, IEEE, 2021, pp. 249–256.

[28] H. Zhou, M. Erol-Kantarci, V. Poor, Learning from peers: Transfer reinforcement learning for joint radio and cache resource allocation in 5G network slicing, 2021, arXiv preprint arXiv:2109.07999.

[29] H. Zhou, M. Erol-Kantarci, Knowledge transfer based radio and computation resource allocation for 5G RAN slicing, in: 2022 IEEE 19th Annual Consumer Communications & Networking Conference, CCNC, IEEE, 2022, pp. 617–623.

[30] R. Ferrús, J. Pérez-Romero, O. Sallent, I. Vilà, R. Agustí, Machine learning-assisted cross-slice radio resource optimization: Implementation framework and algorithmic solution, 2020.

[31] Y. Cui, X. Huang, P. He, D. Wu, R. Wang, A two-timescale resource allocation scheme in vehicular network slicing, in: 2021 IEEE 93rd Vehicular Technology Conference, VTC2021-Spring, IEEE, 2021, pp. 1–5.

[32] M. Mohsenivatani, M. Darabi, S. Parsaeefard, M. Ardebilipour, B. Maham, Throughput maximization in C-RAN enabled virtualized wireless networks via multi-agent deep reinforcement learning, in: 2020 IEEE 31st Annual International Symposium on Personal, Indoor and Mobile Radio Communications, IEEE, pp. 1–6.

[33] M. Yan, G. Feng, J. Zhou, Y. Sun, Y.-C. Liang, Intelligent resource scheduling for 5G radio access network slicing, IEEE Trans. Veh. Technol. 68 (8) (2019) 7691–7703.

[34] Y. Azimi, S. Yousefi, H. Kalbkhani, T. Kunz, Energy-efficient deep reinforcement learning assisted resource allocation for 5G-RAN slicing, IEEE Trans. Veh. Technol. (2021).

[35] M. Yan, B. Chen, G. Feng, S. Qin, Federated cooperation and augmentation for power allocation in decentralized wireless networks, IEEE Access 8 (2020) 48088–48100.

[36] S. Messaoud, A. Bradai, O.B. Ahmed, P. Quang, M. Atri, M.S. Hossain, Deep federated Q-learning-based network slicing for industrial IoT, IEEE Trans. Ind. Inform. (2020).

[37] Q. Liu, N. Choi, T. Han, OnSlicing: online end-to-end network slicing with reinforcement learning, in: Proceedings of the 17th International Conference on Emerging Networking EXperiments and Technologies, 2021, pp. 141–153.

[38] O. Alliance, O-RAN Use Cases and Deployment Scenarios, White Paper, 2020.

[39] B. Liu, P. Zhu, J. Li, D. Wang, X. You, Energy-efficient optimization in distributed massive MIMO systems for slicing eMBB and URLLC services, IEEE Trans. Veh. Technol. (2023).

[40] A. Mei, J. Stefa, SWIM: A simple model to generate small mobile worlds, in: IEEE INFOCOM 2009, IEEE, 2009, pp. 2106–2113.

[41] T. Wang, T. Liang, J. Li, W. Zhang, Y. Zhang, Y. Lin, Adaptive traffic signal control using distributed marl and federated learning, in: 2020 IEEE 20th International Conference on Communication Technology, ICCT, IEEE, 2020, pp. 1242–1248.

[42] S. Lee, D.-H. Choi, Federated reinforcement learning for energy management of multiple smart homes with distributed energy resources, IEEE Trans. Ind. Inform. 18 (1) (2020) 488–497.

[43] P. Izmailov, D. Podoprikhin, T. Garipov, D. Vetrov, A.G. Wilson, Averaging weights leads to wider optima and better generalization, 2018, arXiv preprint arXiv:1803.05407.

[44] A. Udugama, B. Khalilov, A.B. Muslim, A. Förster, Implementation of the swim mobility model in omnet++, 2016, arXiv preprint arXiv:1609.05199.